

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



BÀI TẬP THỰC HÀNH 1B
MÃ HOÁ HIỆN ĐẠI, CHỨNG
THỰC VÀ CHỮ KÝ SỐ

Mã Môn: IE105
Tên Môn: Nhập Môn Đảm Bảo và An Ninh Thông Tin
Lớp: IE105.F22.LT.CNTT
Giảng Viên: ThS. Tô Nguyễn Nhật Quang
Sinh Viên: Đinh Xuân Sâm
Mã Sinh Viên 25410291

Ngày 15 Tháng 04 Năm 2026

TÓM TẮT NỘI DUNG

MỤC TIÊU

- Hiểu nguyên lý mã hoá khóa đối xứng (AES, DES), khóa công khai (RSA), hàm băm (SHA, MD5), chữ ký số.
- Sử dụng CrypTool 1 để mã hoá/giải mã, tạo và kiểm tra chữ ký số, sinh khóa RSA.
- Thực hành kiểm tra tính toàn vẹn bằng hash.
- Viết báo cáo phân tích dựa trên kết quả thực hành.

LÝ THUYẾT NỀN TẢNG

1. Mã hoá khóa đối xứng

- Dùng một khóa duy nhất để mã hoá và giải mã.
- Thuật toán: AES, DES, 3DES.
- Nhanh nhưng khó phân phối khóa.

2. Mã hoá khóa công khai (RSA)

- Dùng 2 khóa (public/private).
- Dùng trong chữ ký số và trao đổi khóa.

3. Hàm băm (Hash)

- Tạo chuỗi dài cố định (128–256 bit).
- Không thể đảo ngược.
- Dùng để kiểm tra toàn vẹn dữ liệu.

4. Chữ ký số

- Private key dùng để ký.
- Public key dùng để xác minh.
- Dùng để xác thực và chống chối bỏ.

CHUẨN BỊ

- Máy Windows 10/11.
- Cài phần mềm CrypTool 1.
- Tạo sẵn các đoạn văn bản để thử nghiệm.

CÁC BƯỚC THỰC HÀNH

Bước 1. Mã hoá & Giải mã bằng mã hoá đối xứng (AES/DES)

- Nhập văn bản: THIS IS A SECRET MESSAGE USED FOR SYMMETRIC ENCRYPTION DEMO.

- Encrypt → Symmetric (modern) → AES → nhập key: MyStrongKey123.
- Chọn mode ECB hoặc thay đổi CBC/CFB/OFB để quan sát khác biệt.
- Giải mã bằng Decrypt → Symmetric (modern) → AES với cùng key.

Bước 2. Hash (Kiểm tra toàn vẹn dữ liệu)

- Analysis → Hashes → chọn MD5, SHA-1, SHA-256 để tạo hash.
- Thay đổi 1 ký tự trong văn bản và tạo hash lại.
- Quan sát hiệu ứng Avalanche: hash thay đổi hoàn toàn.

Bước 3. Mã hoá khóa công khai RSA

- Encrypt → Asymmetric → RSA → Generate Keys → chọn 1024 bit.
- Dùng Public Key để mã hoá văn bản.
- Dùng Private Key để giải mã.
- Thử dùng key 2048 bit và ghi nhận thay đổi tốc độ.

Bước 4. Chữ ký số (Digital Signature)

- Digital Signatures → Create/Verify Signature → RSA Signature.
- Chọn văn bản + Private Key để tạo file chữ ký .sig.
- Verify Signature → chọn văn bản gốc + file .sig + Public Key.
- Thử sửa văn bản rồi kiểm tra lại chữ ký (kết quả INVALID).

Bước 5. Kiểm tra tính toàn vẹn & xác thực

- Người gửi gửi văn bản + hash.
- Người nhận tạo hash mới rồi so sánh.
- Nếu hash không khớp → dữ liệu bị thay đổi.

CÂU HỎI BÁO CÁO

1. Khác nhau giữa mã hoá đối xứng và bất đối xứng?
2. Hash dùng để làm gì? Vì sao không thể đảo ngược?
3. Vì sao sửa 1 ký tự khiến hash đổi toàn bộ?
4. Tại sao chữ ký số cần private key?
5. RSA có dùng để mã hoá file lớn không? Vì sao?
6. Chữ ký số khác mã hoá ở điểm nào?
7. Nhược điểm của mode ECB trong AES?

TỔNG KẾT

- Sinh viên nắm quy trình mã hoá hiện đại.
- Biết sử dụng RSA để mã hoá và ký.
- Thực hành hash để kiểm tra toàn vẹn.
- Hiểu vai trò của khóa công khai & bí mật.

- Áp dụng CrypTool vào an toàn thông tin.

LỜI NÓI ĐẦU

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

LỜI CẢM ƠN

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Giảng Viên

Mục Lục

Mục Lục	vii
Danh Sách Bảng	ix
Danh Sách Hình Ảnh	x
Danh Sách Mã Nguồn	xii
Bảng Viết Tắt	xiii
1. Bước 1. Mã Hoá & Giải Mã Bằng Mã Hoá Đối Xứng (AES/DES)	1
1.1. Chuẩn Bị	1
1.2. Mã Hóa AES (CBC)	2
1.3. Giải Mã AES (CBC)	3
1.4. Mã Hóa Triple DES (CBC)	4
1.5. Giải Mã Triple DES (CBC)	5
1.6. So Sánh Văn Bản Mã Hóa AES (CBC) vs Triple DES (CBC)	6
2. Bước 2. Hash (Kiểm Tra Toàn Vẹn Dữ Liệu)	7
2.1. Văn Bản Gốc	7
2.2. Hash	7
2.3. Thay Đổi Văn Bản Và Thử Lại	8
3. Bước 3. Mã Hoá Khóa Công Khai RSA	10
3.1. Văn Bản Gốc	10
3.2. Tạo Cặp Khóa RSA	10
3.3. Mã Hóa Văn Bản Với Public Key	13
3.4. Giải Mã Với Private Key	15
3.5. Thử Nghiệm Với Độ Dài Bit 2048	16
4. Bước 4. Chữ Ký Số (Digital Signature)	21
4.1. Sign Document	21
4.2. Verify Signature	22
4.3. Giả Lập Chữ Ký Sai	24
5. Bước 5. Kiểm Tra Tính Toàn Vẹn & Xác Thực	26
5.1. Văn Bản Gốc Và Hash	26
5.2. Trường Hợp 1: Văn Bản Nguyên Vẹn	26
5.3. Trường Hợp 2: Văn Bản Bị Thay Đổi	27
5.4. Demo	27
6. CÂU HỎI BÁO CÁO	28
7. TỔNG KẾT	30
A Thông Tin Về Báo Cáo Đây	31
A.1 Mã Nguồn Báo Cáo	31
A.2 Mã Nguồn Nội Dung Chính	31

Tài Liệu Tham Khảo	32
--------------------------	----

DANH SÁCH BẢNG

DANH SÁCH HÌNH ẢNH

Hình ảnh 1	Bước 1.1. Văn Bản Gốc	1
Hình ảnh 2	Bước 1.2. Key MyStrongKey123	2
Hình ảnh 3	Bước 1.3. Mã Hóa AES (CBC)	3
Hình ảnh 4	Bước 1.4. Văn Bản Mã Hóa	3
Hình ảnh 5	Bước 1.5. Giải Mã	4
Hình ảnh 6	Bước 1.6. Mã Hóa Triple DES (CBC)	4
Hình ảnh 7	Bước 1.7. Văn Bản Mã Hóa Triple DES (CBC)	5
Hình ảnh 8	Bước 1.8. Giải Mã Triple DES (CBC)	5
Hình ảnh 9	Bước 1.9. So Sánh Văn Bản Mã Hóa AES (CBC) vs Triple DES (CBC)	6
Hình ảnh 10	Bước 2.1. Văn Bản Gốc	7
Hình ảnh 11	Bước 2.2. Hashes	8
Hình ảnh 12	Bước 2.3. Hashes Sau Khi Đổi 1 Ký Tự	9
Hình ảnh 13	Bước 3.1. Tạo Cặp Khóa RSA	11
Hình ảnh 14	Bước 3.2. Tạo Cặp Khóa RSA	11
Hình ảnh 15	Bước 3.3. Tạo Cặp Khóa RSA	12
Hình ảnh 16	Bước 3.4. Show Key Pair	13
Hình ảnh 17	Bước 3.5. Export PSE (PKCS#12)	13
Hình ảnh 18	Bước 3.6. Mã Hóa Văn Bản Với Public Key	14
Hình ảnh 19	Bước 3.7. Văn Bản Mã Hóa	14
Hình ảnh 20	Bước 3.8. Giải Mã Văn Bản Với Private Key	15
Hình ảnh 21	Bước 3.9. Văn Bản Giải Mã	16
Hình ảnh 22	Bước 3.10. Thông Báo Sai Mã PIN	16
Hình ảnh 23	Bước 3.11. Tạo Cặp Khóa RSA 2048 Bit	17
Hình ảnh 24	Bước 3.12. Thông Báo Tạo Cặp Khóa RSA 2048 Bit	17
Hình ảnh 25	Bước 3.13. Show Key Pair	18
Hình ảnh 26	Bước 3.14. Mã Hóa Văn Bản Với Public Key 2048 Bit	18
Hình ảnh 27	Bước 3.15. Giải Mã Văn Bản Với Private Key 2048 Bit	19
Hình ảnh 28	Bước 3.16. Văn Bản Giải Mã	19
Hình ảnh 29	Bước 4.1. Tạo Chữ Ký Số	21
Hình ảnh 30	Bước 4.2. Văn Bản Đã Ký Và Chữ Ký	22
Hình ảnh 31	Bước 4.3. Xem Chữ Ký	22
Hình ảnh 32	Bước 4.4. Xác Nhận Chữ Ký	23
Hình ảnh 33	Bước 4.5. Kết Quả Xác Nhận Chữ Ký	24
Hình ảnh 34	Bước 4.6. Giả Lập Chữ Ký Sai	24
Hình ảnh 35	Bước 4.7. Chạy Lại Xác Nhận Chữ Ký Sai	25

Hình ảnh 36	Bước 4.8. Kết Quả Xác Nhận Chữ Ký Sai	25
Hình ảnh 37	Bước 5. Kiểm Tra Tính Toàn Vẹn & Xác Thực	27

DANH SÁCH MÃ NGUỒN

BẢNG VIẾT TẮT

Viết Tắt	Nghĩa Đầy Đủ
API	Application Programming Interface
CNTT	Công Nghệ Thông Tin
CSDL	Cơ Sở Dữ Liệu
HTML	HyperText Markup Language
UI	User Interface
UX	User Experience

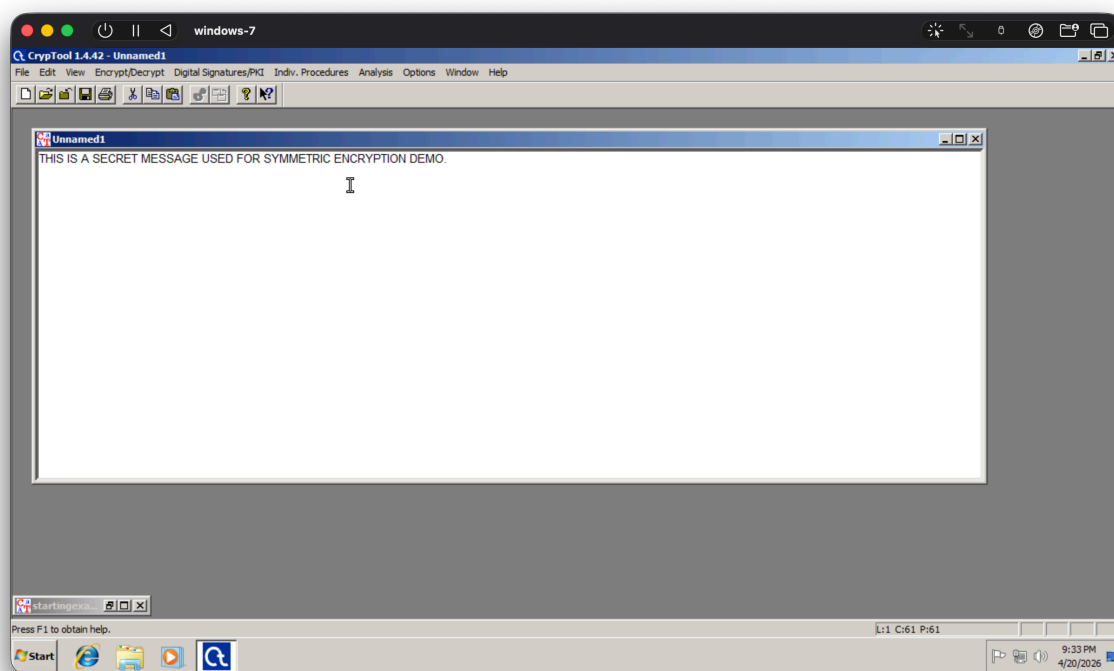
Bảng Viết Tắt

CHƯƠNG 1.

BƯỚC 1. MÃ HOÁ & GIẢI MÃ BẰNG MÃ HOÁ ĐỐI XỨNG (AES/DES)

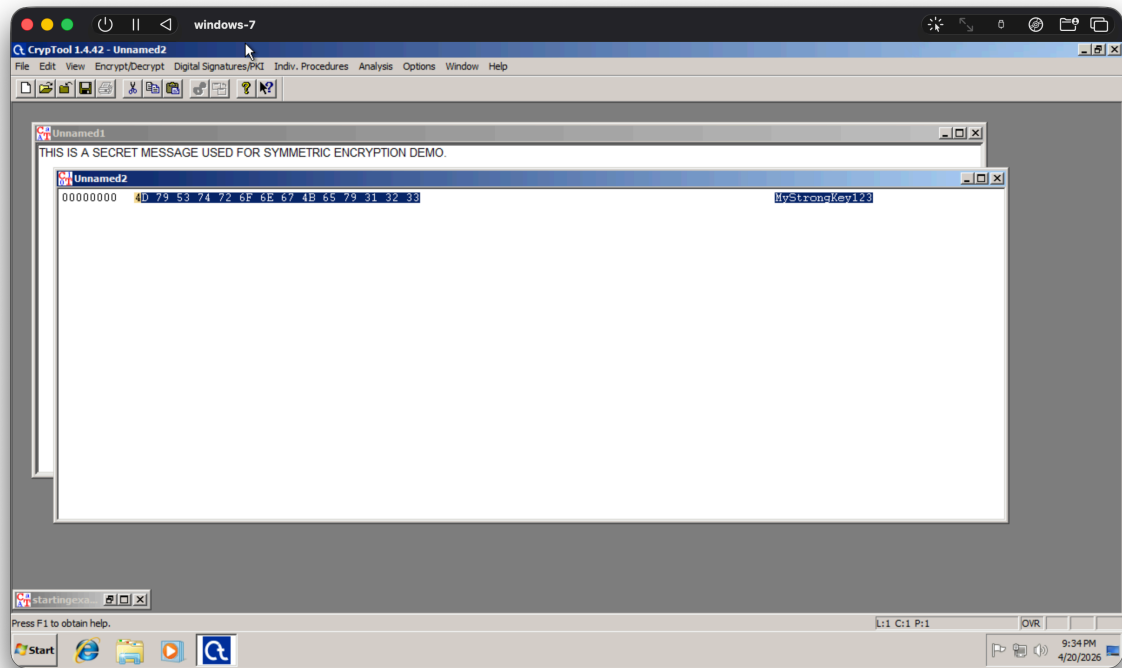
1.1. Chuẩn Bị

- Nhập văn bản: THIS IS A SECRET MESSAGE USED FOR SYMMETRIC ENCRYPTION DEMO.



Hình ảnh 1: Bước 1.1. Văn Bản Gốc

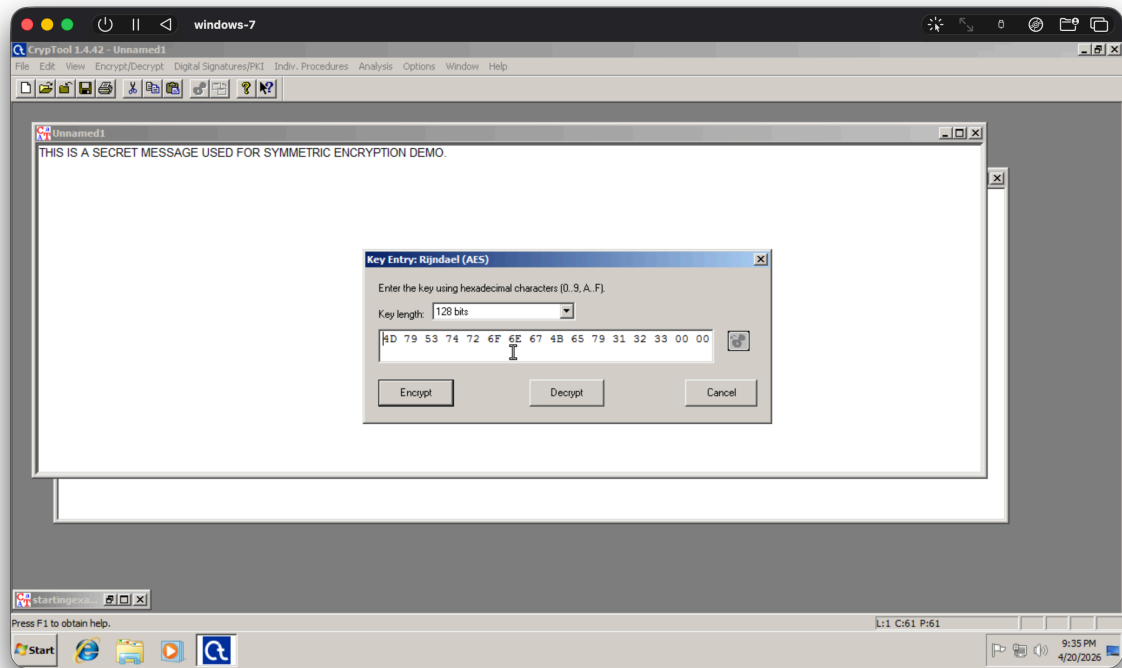
- Key `MyStrongKey123` và dạng Hex:
 - Chọn View > Hex dump



Hình ảnh 2: Bước 1.2. Key MyStrongKey123

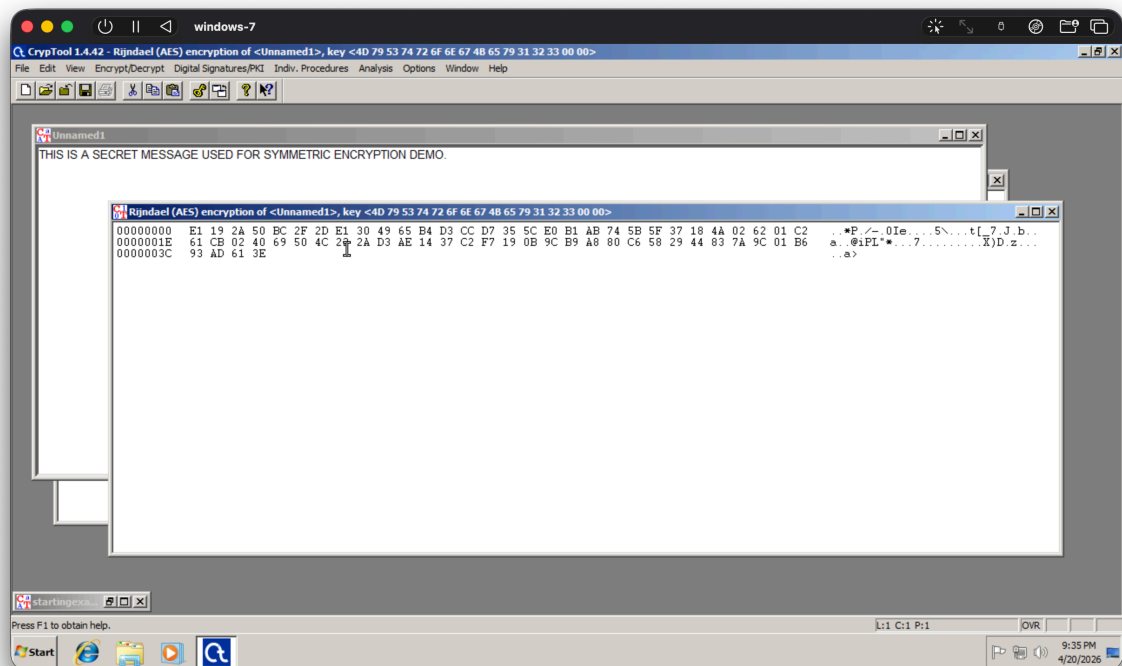
1.2. Mã Hóa AES (CBC)

- Encrypt → Symmetric (modern) → AES (CBC) → nhập key: MyStrongKey123 (dạng Hex)
 - 4D 79 53 74 72 6F 6E 67 4B 65 79 31 32 33
 - Các bit thiếu sẽ được thêm vào ở dạng 00 00



Hình ảnh 3: Bước 1.3. Mã Hóa AES (CBC)

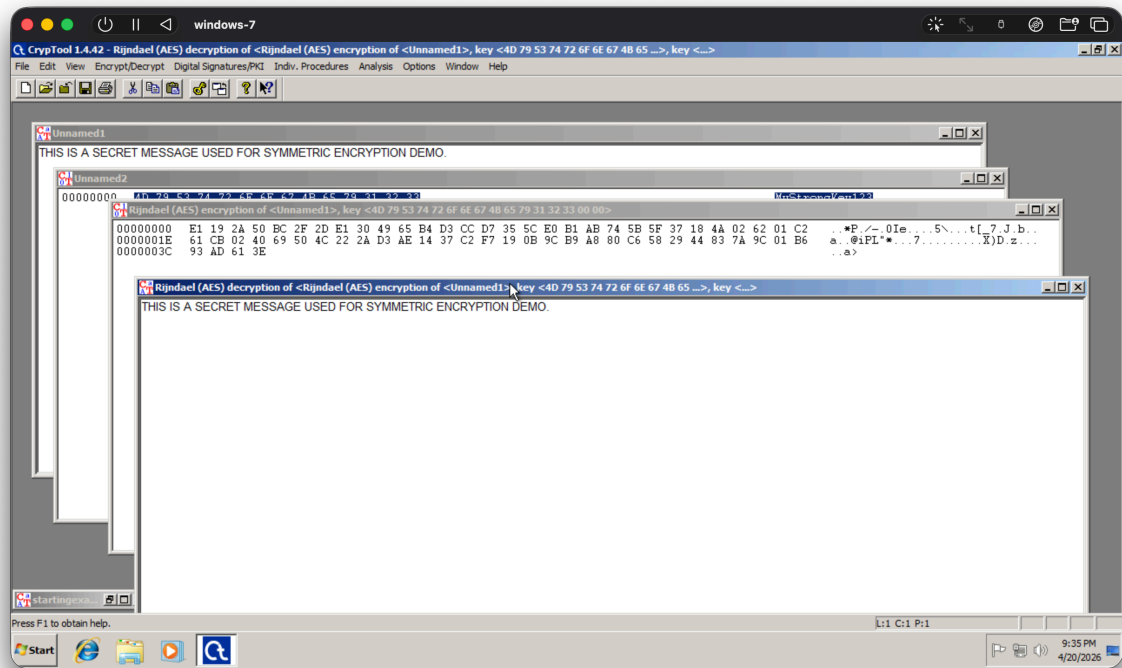
- Văn bản đã được mã hóa:



Hình ảnh 4: Bước 1.4. Văn Bản Mã Hóa

1.3. Giải Mã AES (CBC)

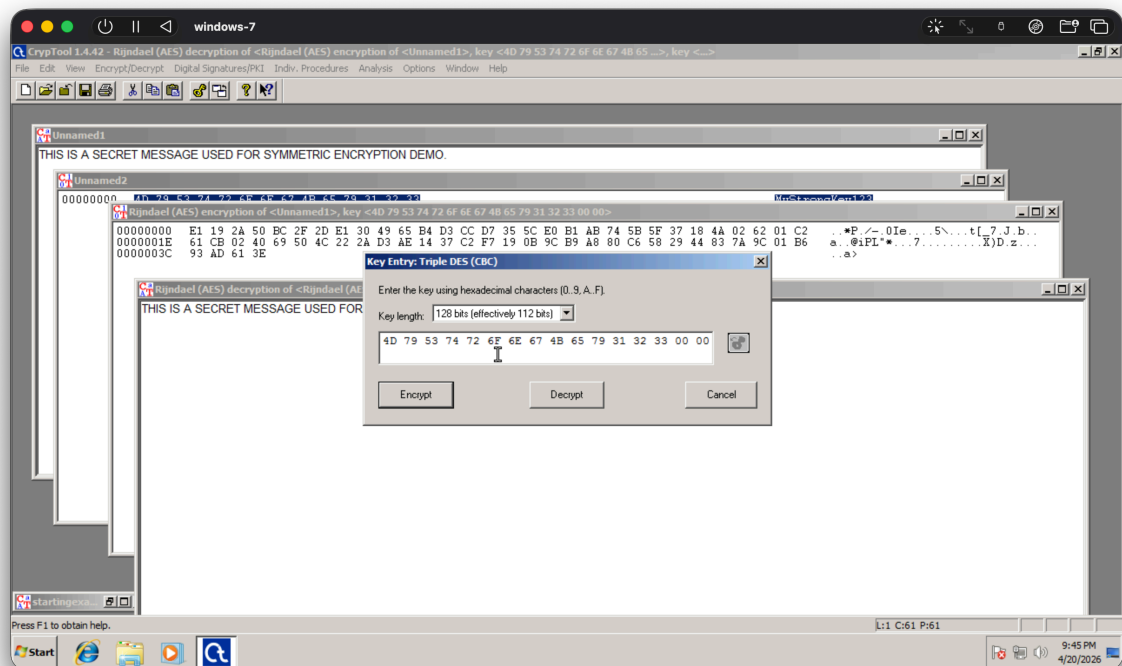
- Giải mã bằng Decrypt → Symmetric (modern) → AES với cùng key.



Hình ảnh 5: Bước 1.5. Giải Mã

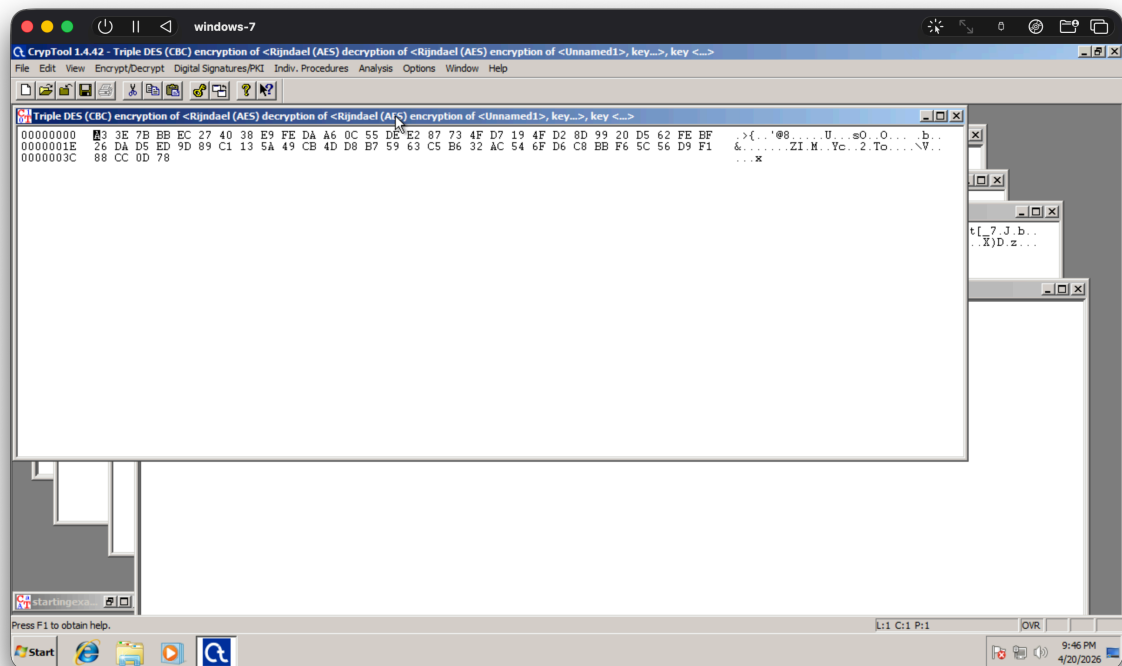
1.4. Mã Hóa Triple DES (CBC)

- Encrypt → Symmetric (modern) → Triple DES (CBC) → nhập key: MyStrongKey123 (dạng Hex)



Hình ảnh 6: Bước 1.6. Mã Hóa Triple DES (CBC)

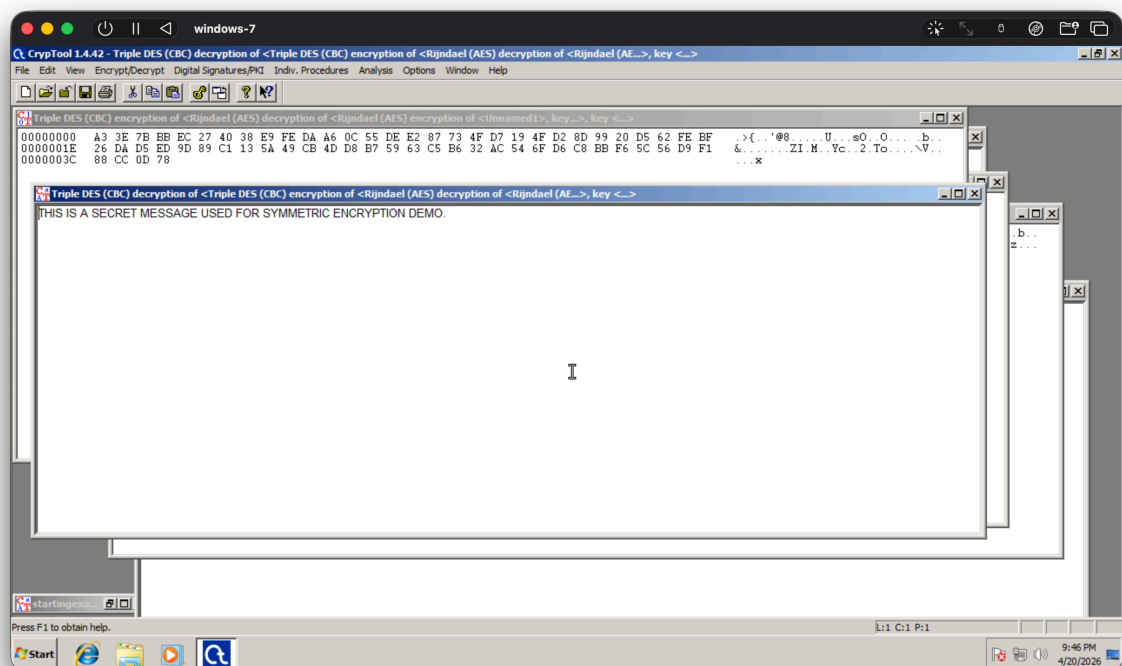
- Văn bản đã được mã hóa:



Hình ảnh 7: Bước 1.7. Văn Bản Mã Hóa Triple DES (CBC)

1.5. Giải Mã Triple DES (CBC)

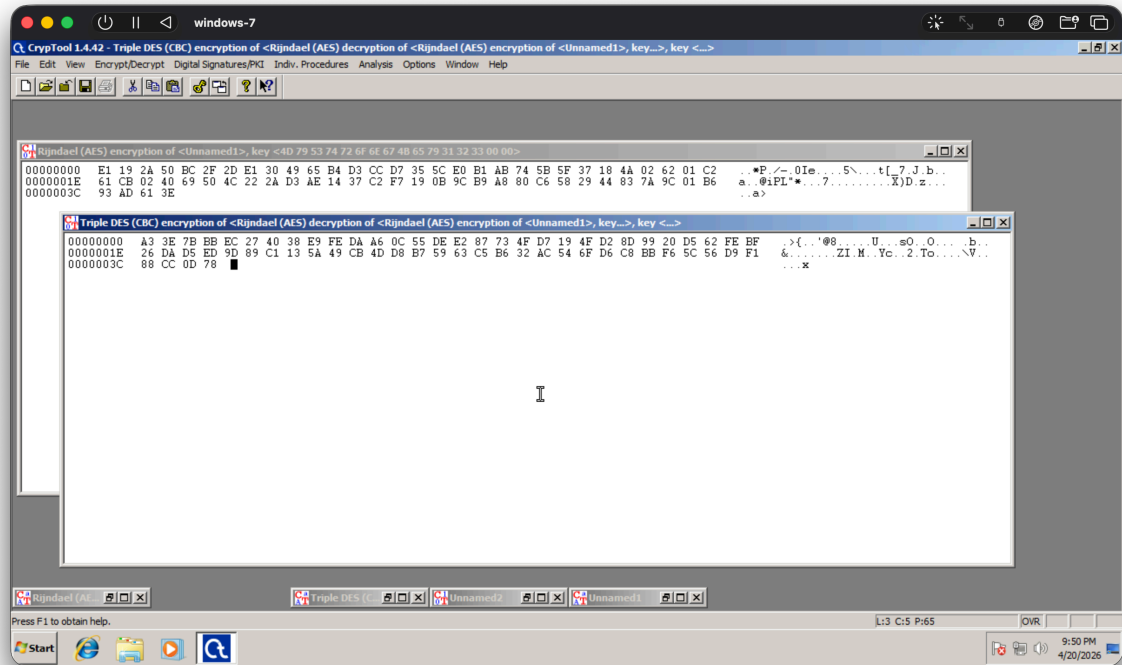
- Giải mã bằng Decrypt → Symmetric (modern) → Triple DES (CBC) với cùng key.



Hình ảnh 8: Bước 1.8. Giải Mã Triple DES (CBC)

1.6. So Sánh Văn Bản Mã Hóa AES (CBC) vs Triple DES (CBC)

- Văn bản mã hóa của AES (CBC) và Triple DES (CBC) có sự khác biệt.



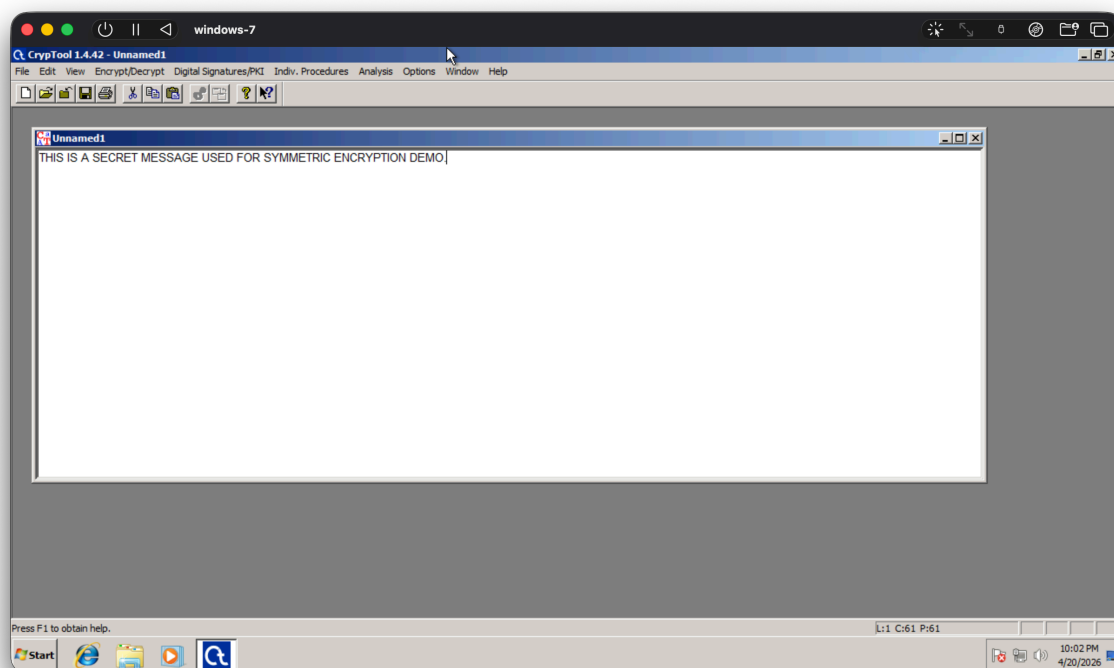
Hình ảnh 9: Bước 1.9. So Sánh Văn Bản Mã Hóa AES (CBC) vs Triple DES (CBC)

CHƯƠNG 2.

BƯỚC 2. HASH (KIỂM TRA TOÀN VỆ DỮ LIỆU)

2.1. Văn Bản Gốc

1 THIS IS A SECRET MESSAGE USED FOR SYMMETRIC ENCRYPTION DEMO.

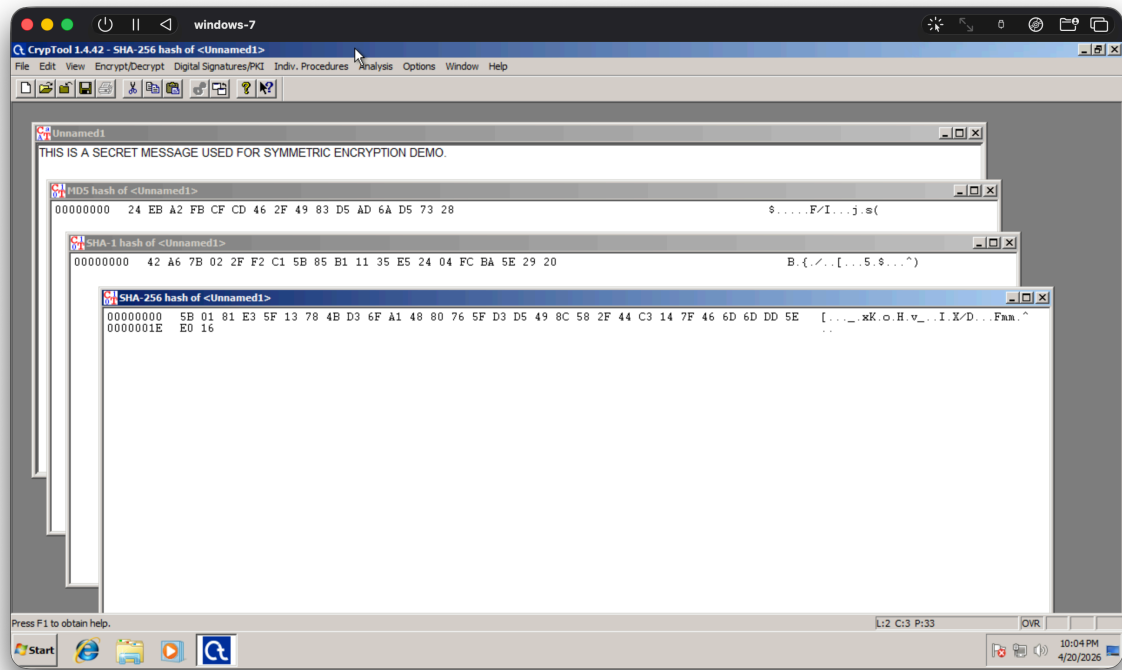


Hình ảnh 10: Bước 2.1. Văn Bản Gốc

2.2. Hash

- Indiv. Procedures → Hashes → chọn MD5, SHA-1, SHA-256 để tạo hash.
 - Các giá trị hash được tạo ra tương ứng.

```
1 - MD5:      24 EB A2 FB CF CD 46 2F 49 83 D5 AD 6A D5 73 28
2 - SHA1:     42 A6 7B 02 2F F2 C1 5B 85 B1 11 35 E5 24 04 FC BA 5E 29 20
3 - SHA256:   5B 01 81 E3 5F 13 78 4B D3 6F A1 48 80 76 5F D3 D5 49 8C 58 2F 44 C3 14 7F 46 6D
              6D DD 5E E0 16
```



Hình ảnh 11: Bước 2.2. Hashes

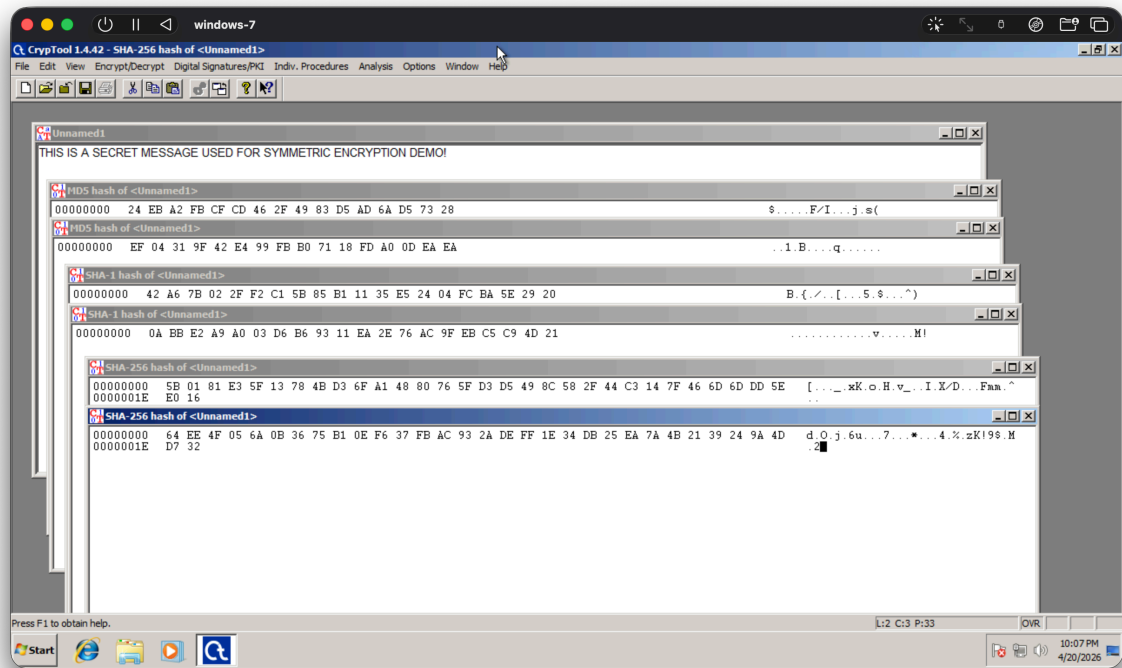
2.3. Thay Đổi Văn Bản Và Thử Lại

- Thay đổi 1 ký tự trong văn bản và tạo hash lại: Đổi dấu '.' thành dấu '!'.

```
1 - THIS IS A SECRET MESSAGE USED FOR SYMMETRIC ENCRYPTION DEMO.
2 + THIS IS A SECRET MESSAGE USED FOR SYMMETRIC ENCRYPTION DEMO!
3 -
```

- Quan sát hiệu ứng Avalanche: hash thay đổi hoàn toàn.
 - Các mã hash với cùng các thuật toán đã dùng cho kết quả khác hoàn toàn.

```
1 - MD5:      24 EB A2 FB CF CD 46 2F 49 83 D5 AD 6A D5 73 28
2 + MD5:      EF 04 31 9F 42 E4 99 FB B0 71 18 FD A0 0D EA EA
3 - SHA1:     42 A6 7B 02 2F F2 C1 5B 85 B1 11 35 E5 24 04 FC BA 5E 29 20
4 + SHA1:     0A BB E2 A9 A0 03 D6 B6 93 11 EA 2E 76 AC 9F EB C5 C9 4D 21
5 - SHA256:   5B 01 81 E3 5F 13 78 4B D3 6F A1 48 80 76 5F D3 D5 49 8C 58 2F 44 C3 14 7F 46 6D
6 6D DD 5E E0 16
+ SHA256:     64 EE 4F 05 6A 0B 36 75 B1 0E F6 37 FB AC 93 2A DE FF 1E 34 DB 25 EA 7A 4B 21 39
24 9A 4D D7 32
```



Hình ảnh 12: Bước 2.3. Hashes Sau Khi Đổi 1 Ký Tự

CHƯƠNG 3.

BƯỚC 3. MÃ HOÁ KHÓA CÔNG KHAI RSA

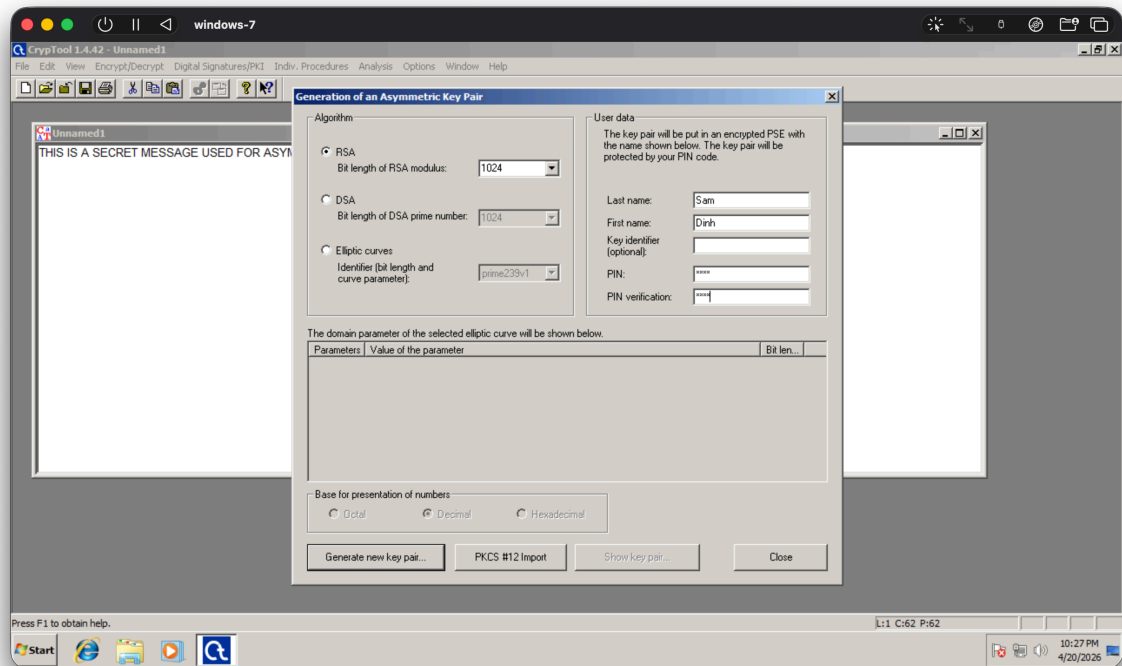
3.1. Văn Bản Gốc

```
1 THIS IS A SECRET MESSAGE USED FOR ASYMMETRIC ENCRYPTION DEMO.
```

3.2. Tạo Cặp Khóa RSA

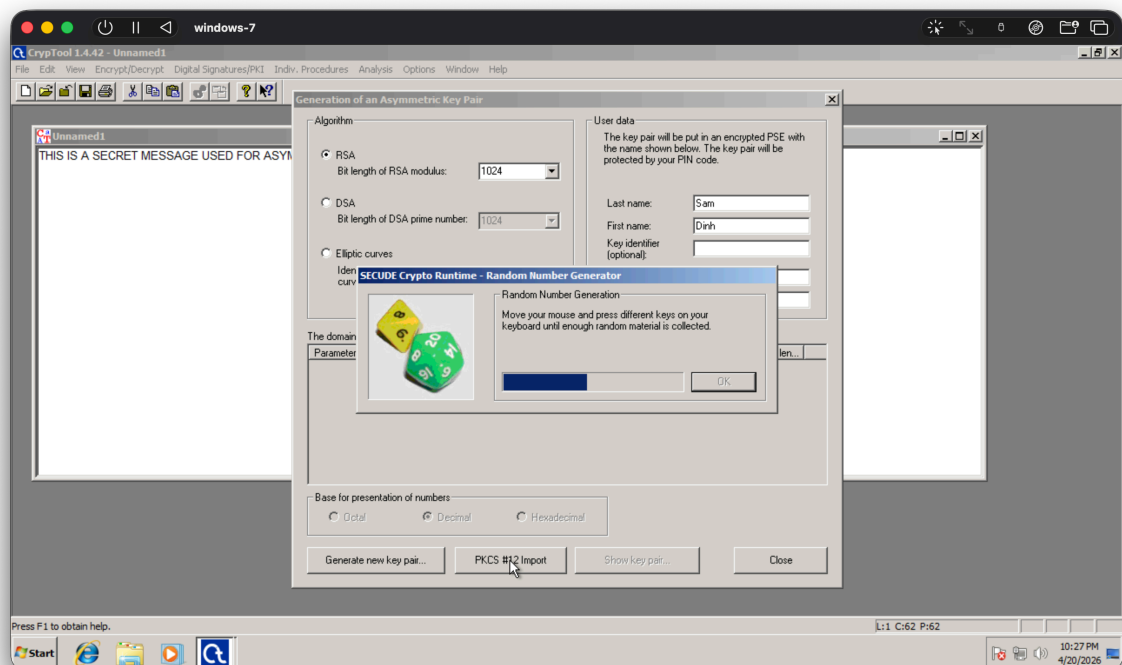
- Digital Signatures/KPI → PKI → Generate/Import Keys...
- Nhập các thông tin cần thiết.
 - Sau đó bấm Tạo Cặp Khóa (**Generate new key pair...**).

```
1 + First Name:      Sam
2 + Last Name:       Dinh
3 + PIN:             1234
4 + PIN verification: 1234
```



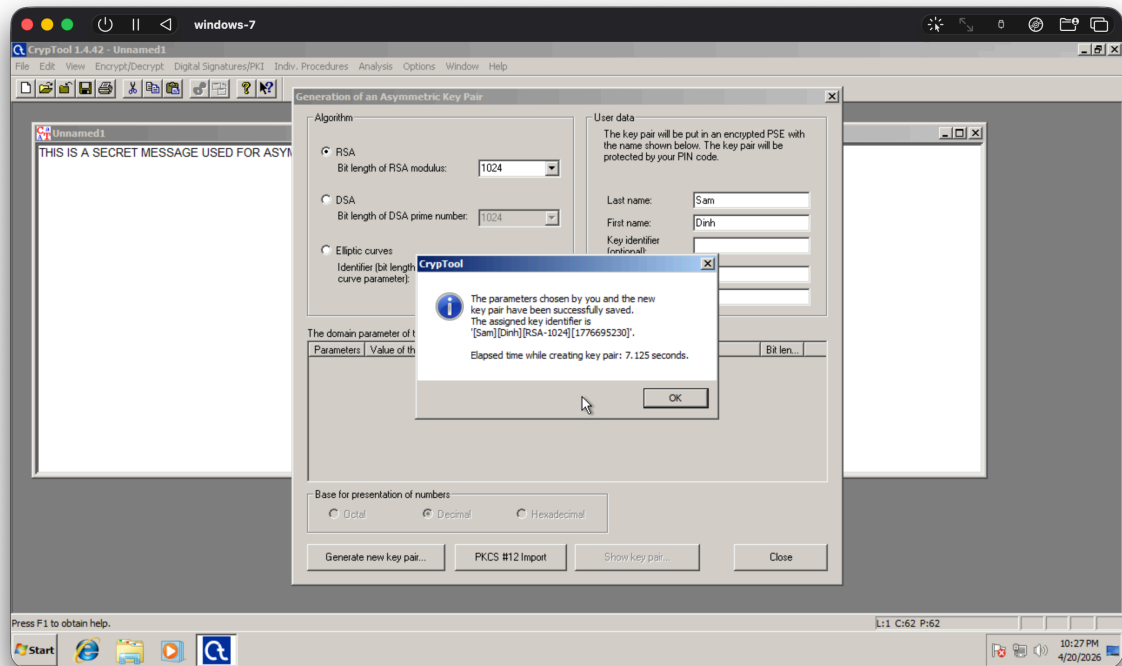
Hình ảnh 13: Bước 3.1. Tạo Cặp Khóa RSA

- Quá trình tạo cặp khóa sẽ bắt đầu:
 - Thời gian hoàn thành vào độ lớn của khóa, sức mạnh tính toán, v...



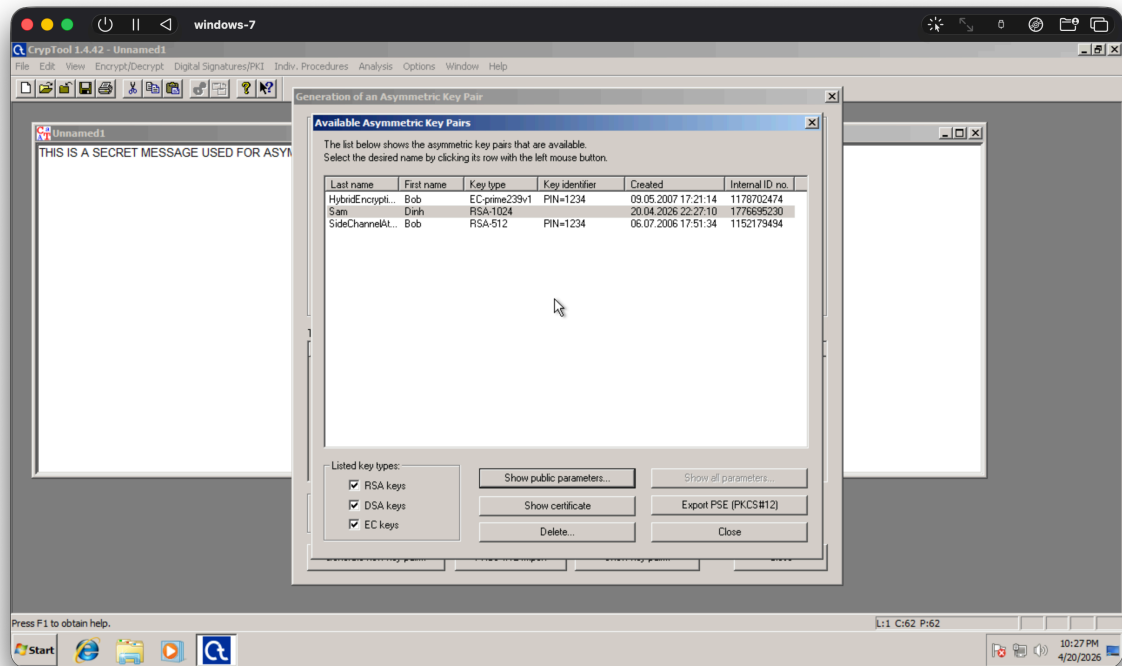
Hình ảnh 14: Bước 3.2. Tạo Cặp Khóa RSA

- Sau khi tạo cặp khóa thành công, ta sẽ có thông báo về việc tạo cặp khóa thành công.
 - Hoàn thành trong khoảng 7.125 giây.



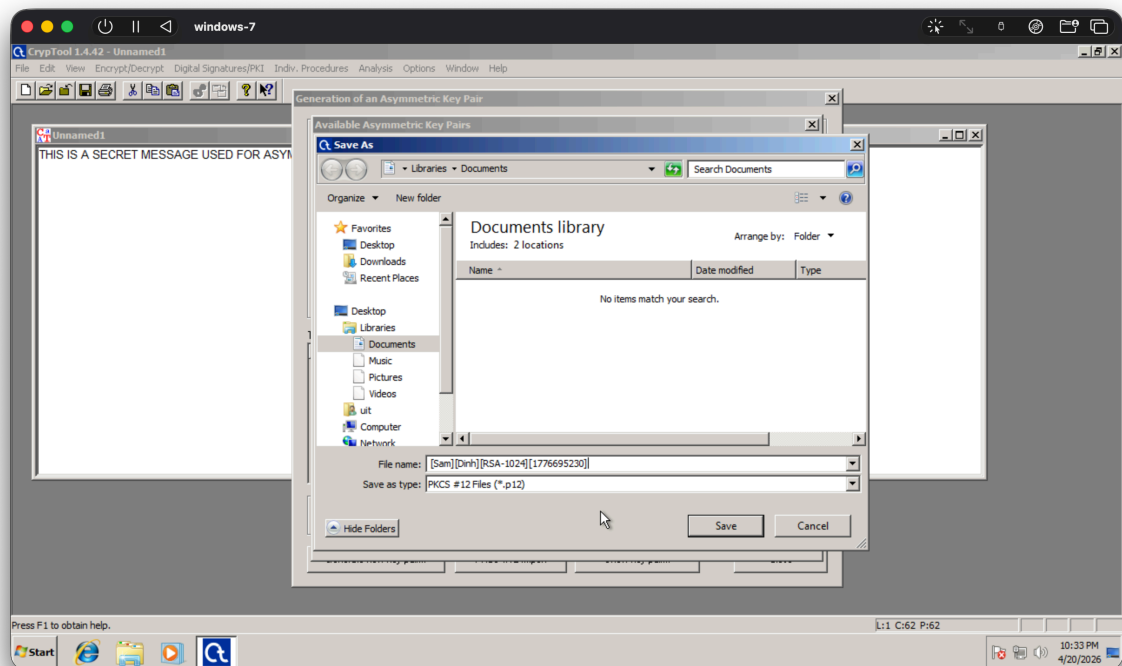
Hình ảnh 15: Bước 3.3. Tạo Cặp Khóa RSA

- Bấm chọn **Show key pair...** để xem cặp khóa vừa tạo.



Hình ảnh 16: Bước 3.4. Show Key Pair

- Có thể chọn **Export PSE (PKCS#12)** để lưu trữ.

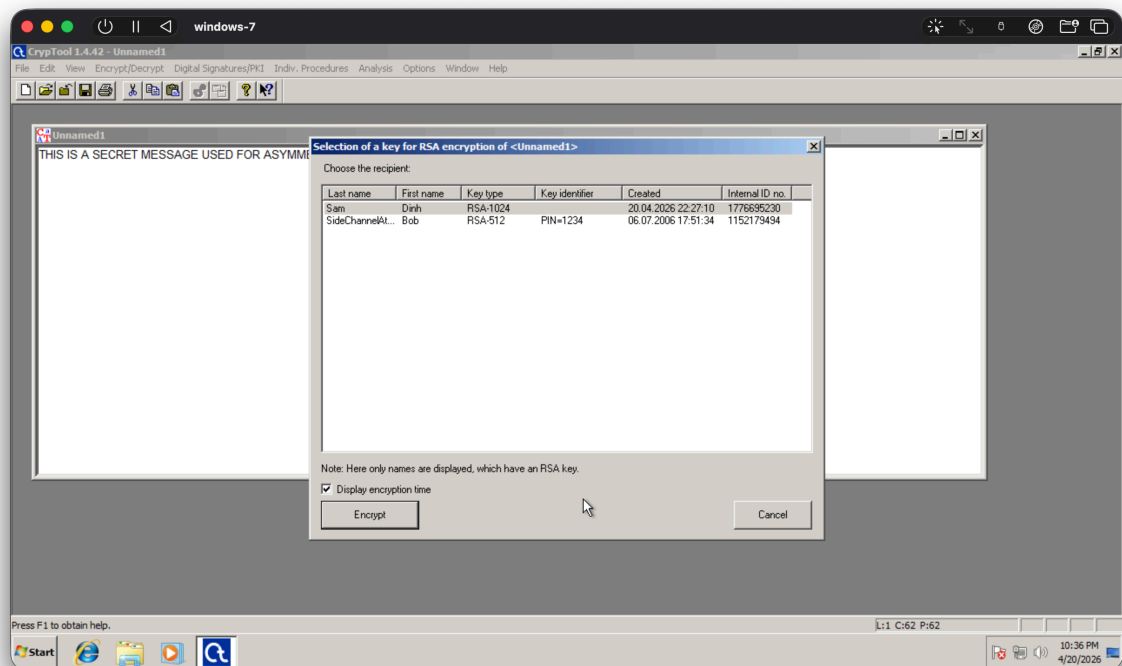


Hình ảnh 17: Bước 3.5. Export PSE (PKCS#12)

3.3. Mã Hóa Văn Bản Với Public Key

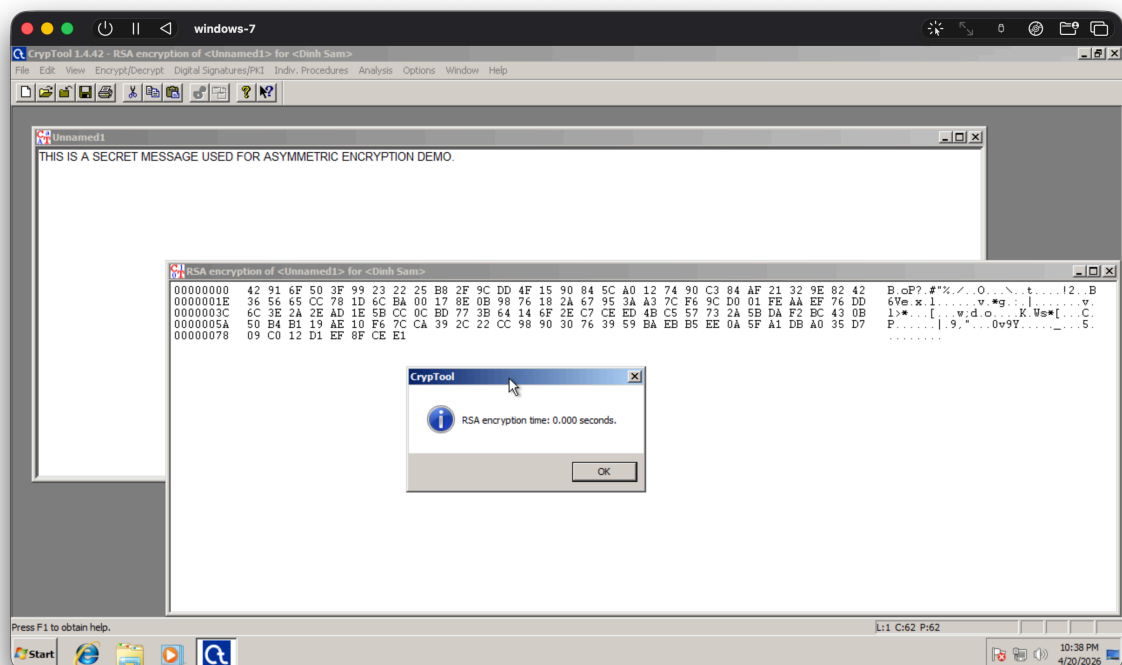
- Chọn **Encrypt/Decrypt** → **Asymmetric** → **RSA** → **Encryption**.

- Chọn key vừa tạo.



Hình ảnh 18: Bước 3.6. Mã Hóa Văn Bản Với Public Key

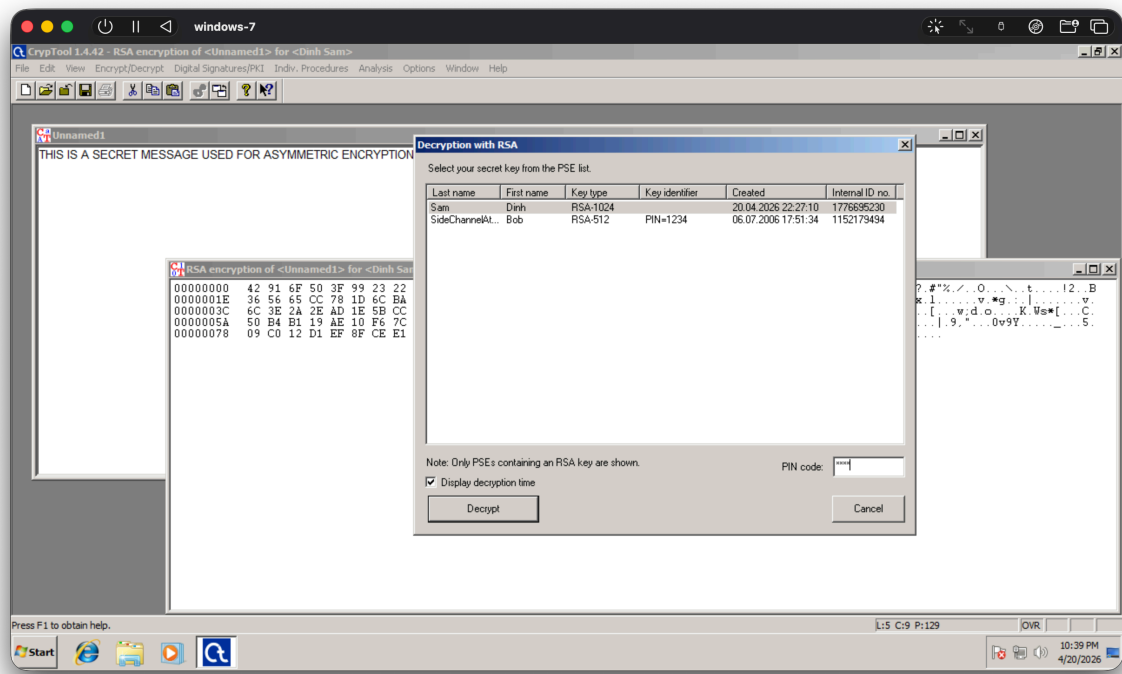
- Mã hóa thành công, trong chưa tới 1 giây



Hình ảnh 19: Bước 3.7. Văn Bản Mã Hóa

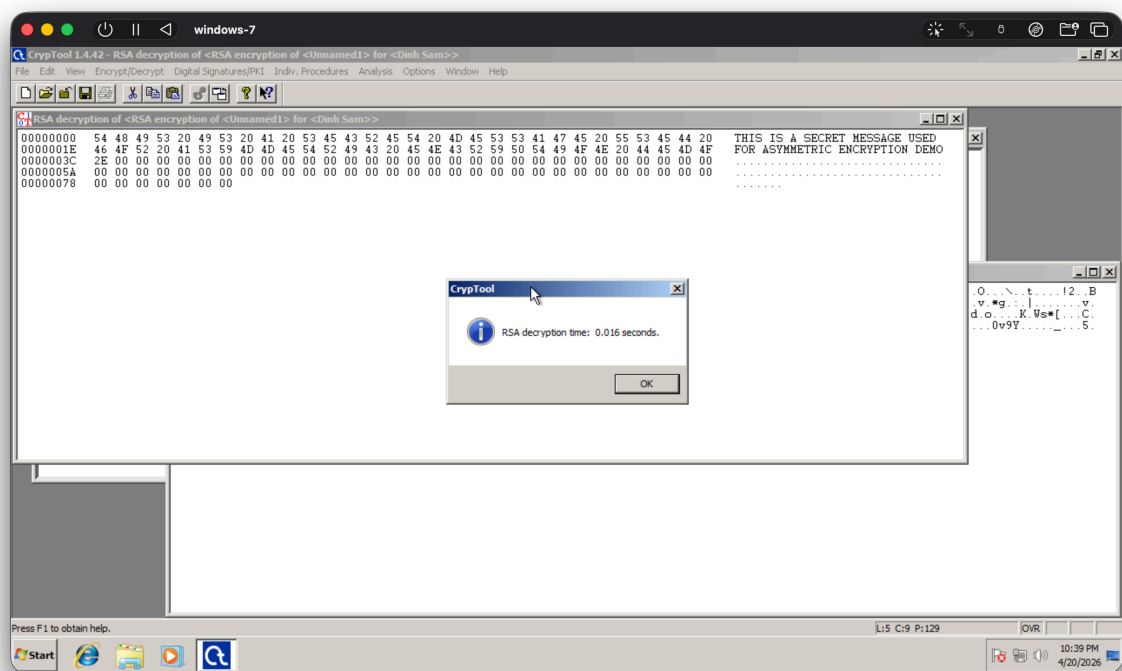
3.4. Giải Mã Với Private Key

- Chọn **Encrypt/Decrypt** → **Asymmetric** → **RSA** → **Decryption**.
 - Chọn key vừa tạo cần dùng.
 - Điền vào mã PIN khi tạo key.



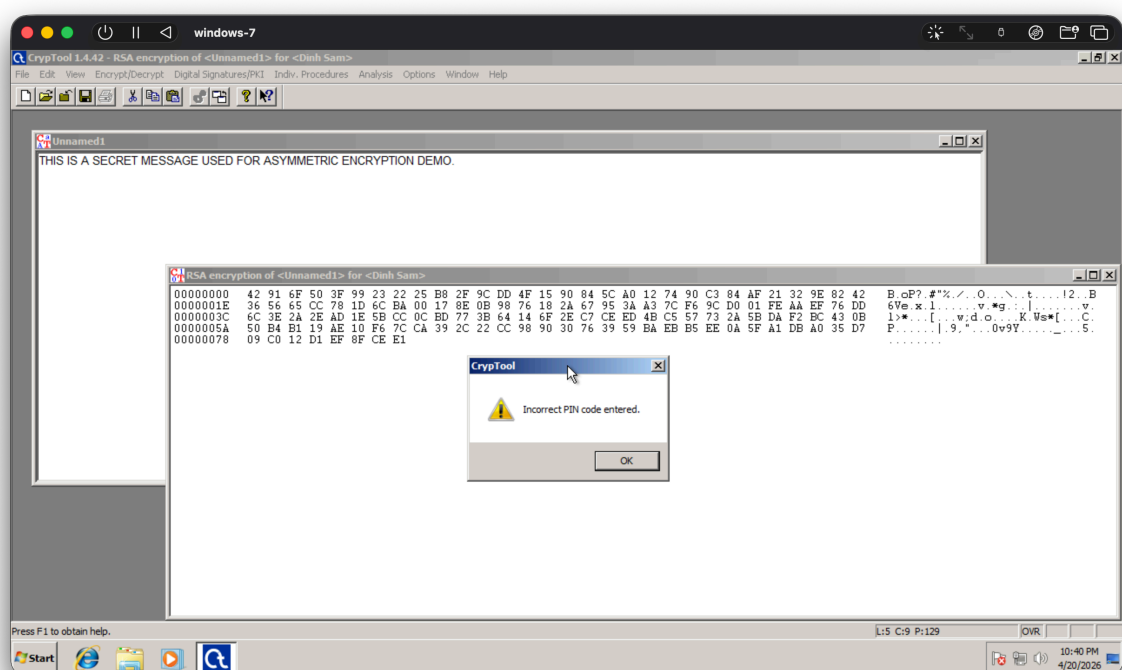
Hình ảnh 20: Bước 3.8. Giải Mã Văn Bản Với Private Key

- Kết quả giải mã:
 - Văn bản gốc ở bên phải.
 - Hoàn thành trong 0.016 giây.



Hình ảnh 21: Bước 3.9. Văn Bản Giải Mã

- Lưu ý: Nếu không điền đúng mã PIN, nhận thông báo lỗi.

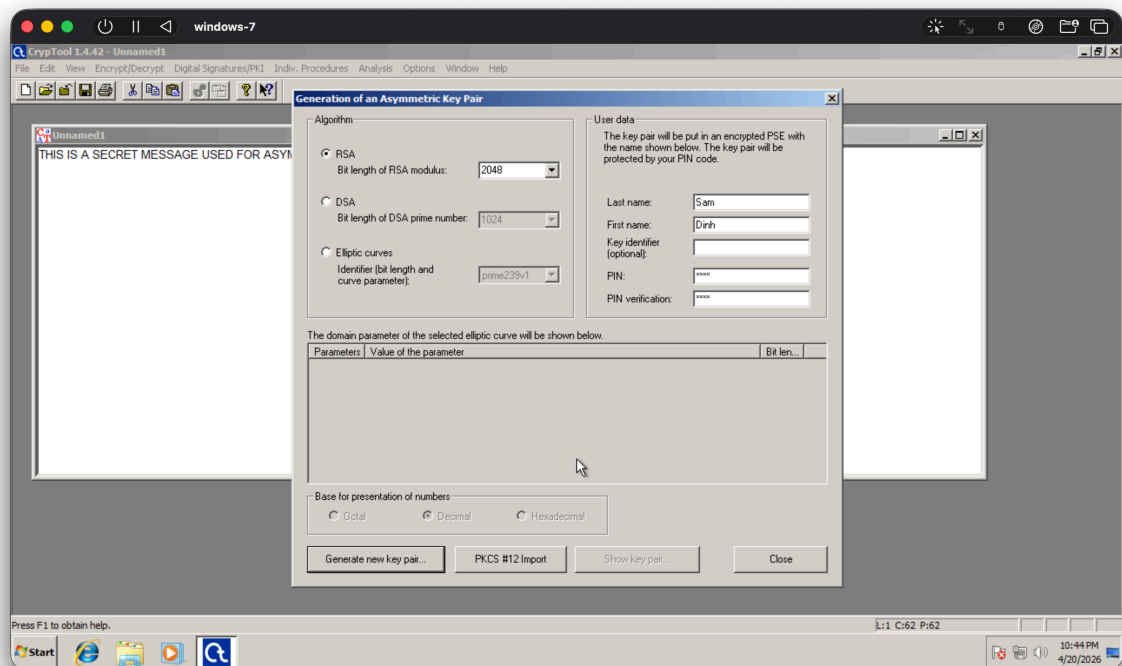


Hình ảnh 22: Bước 3.10. Thông Báo Sai Mã PIN

3.5. Thử Nghiệm Với Độ Dài Bit 2048

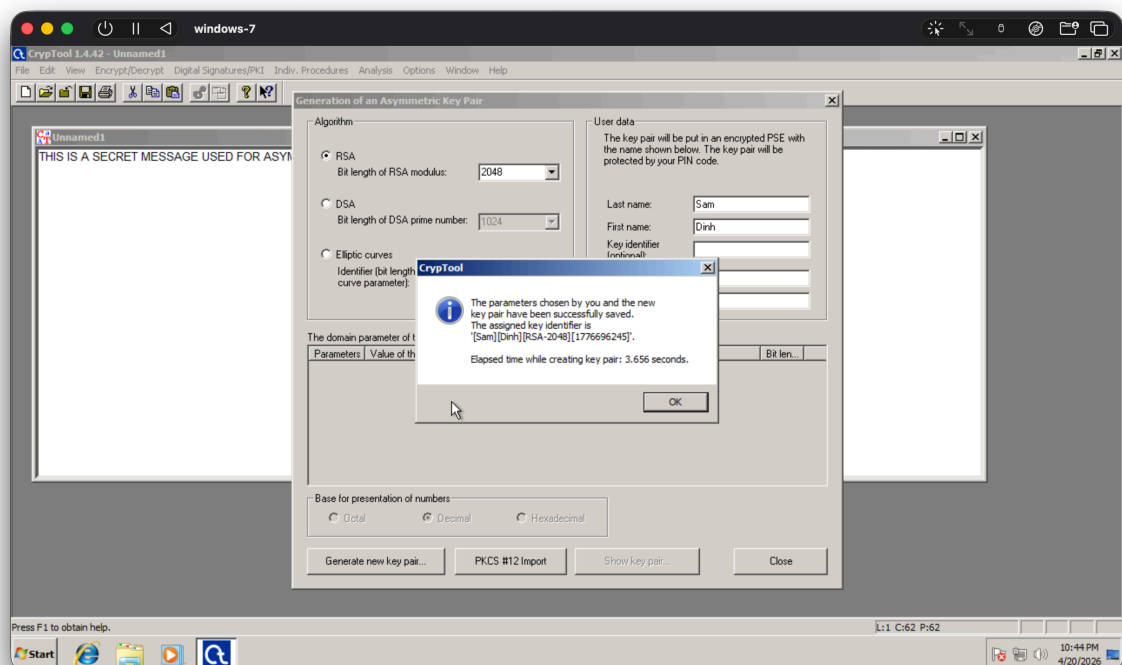
- Tạo mã key mới với cùng thông tin nhưng khác về độ dài bit.

- Độ dài bit: 2048



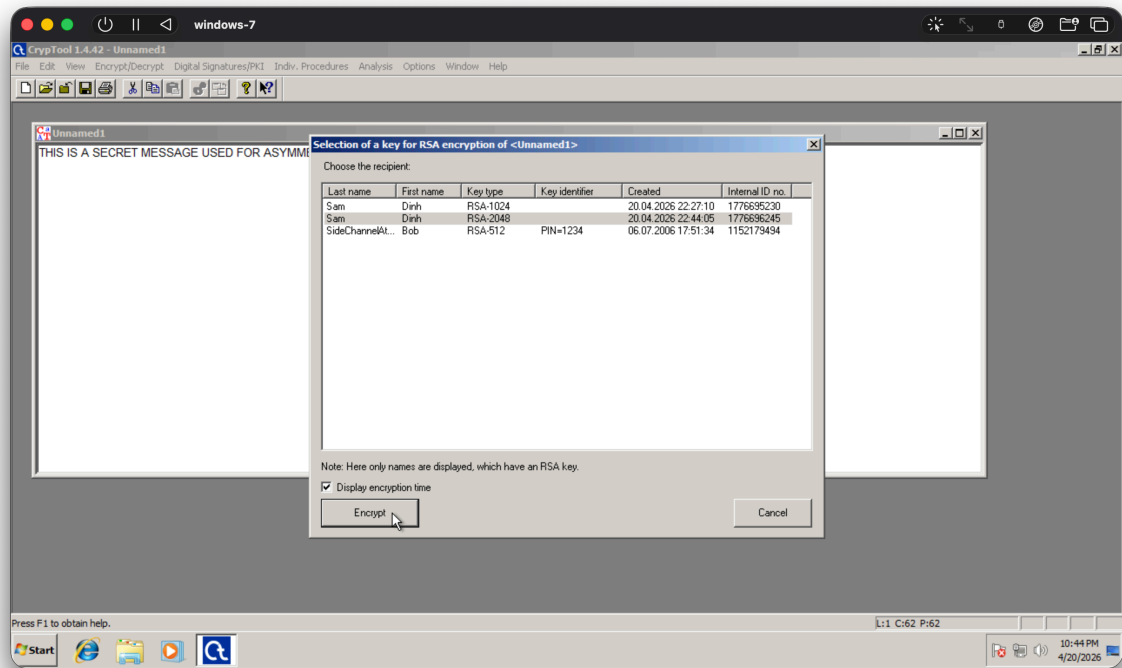
Hình ảnh 23: Bước 3.11. Tạo Cặp Khóa RSA 2048 Bit

- Thông báo tạo cặp key mới hoàn thành: 3.656 giây.



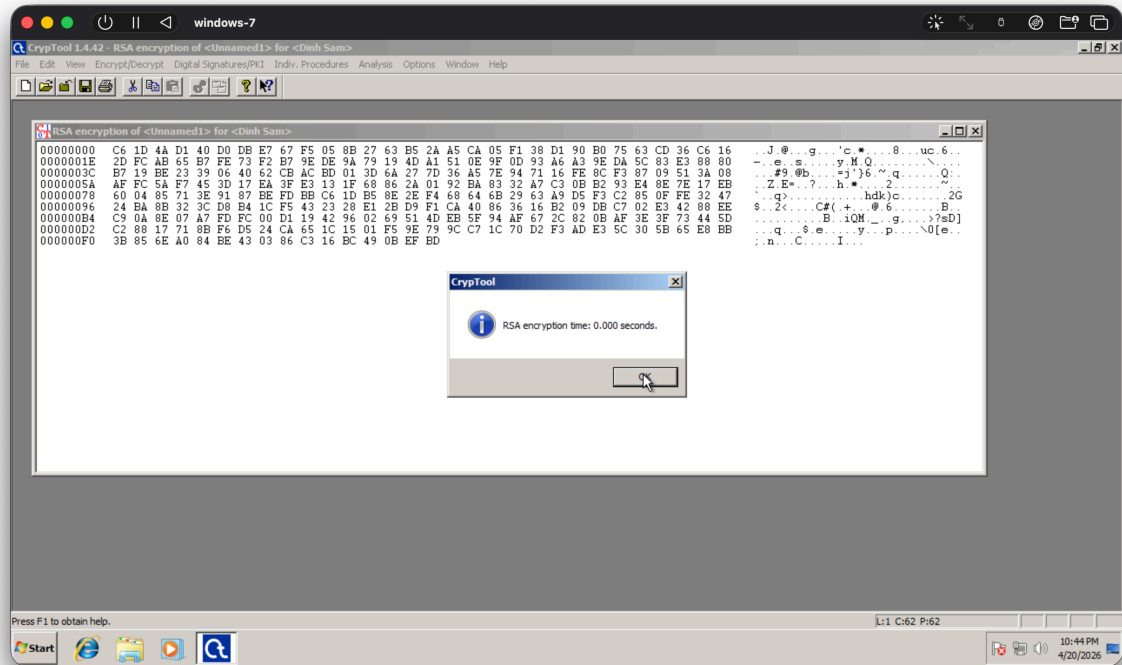
Hình ảnh 24: Bước 3.12. Thông Báo Tạo Cặp Khóa RSA 2048 Bit

- Mã key mới đã sẵn sàng: Sam Dinh RSA-2048



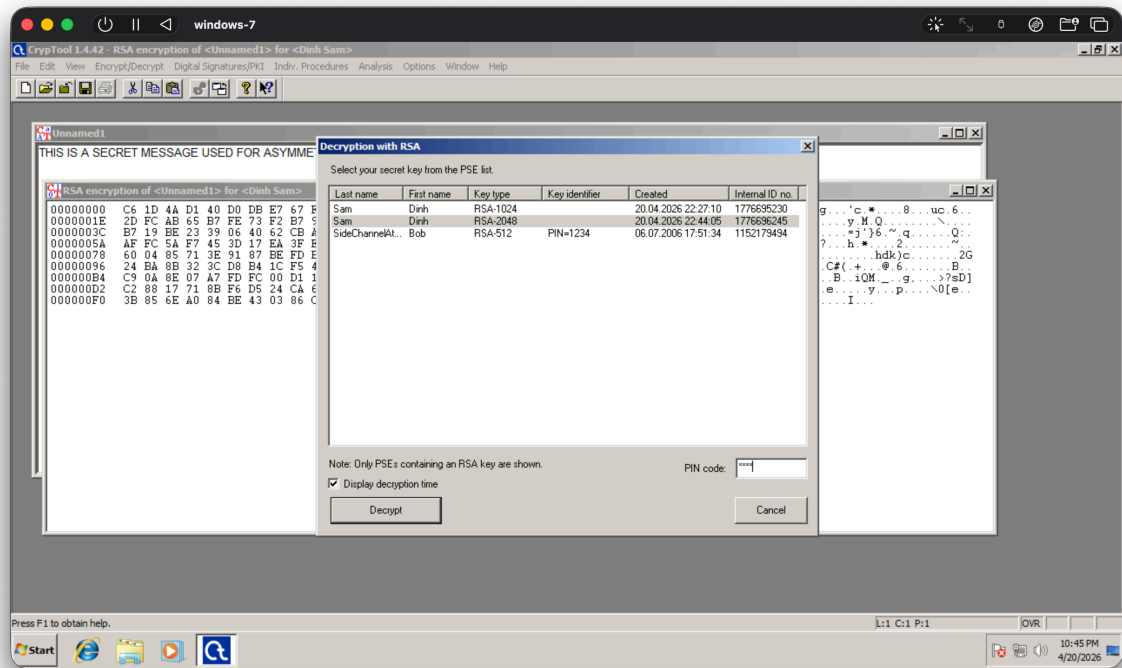
Hình ảnh 25: Bước 3.13. Show Key Pair

- Mã hóa văn bản gốc với key RSA 2048 bit.



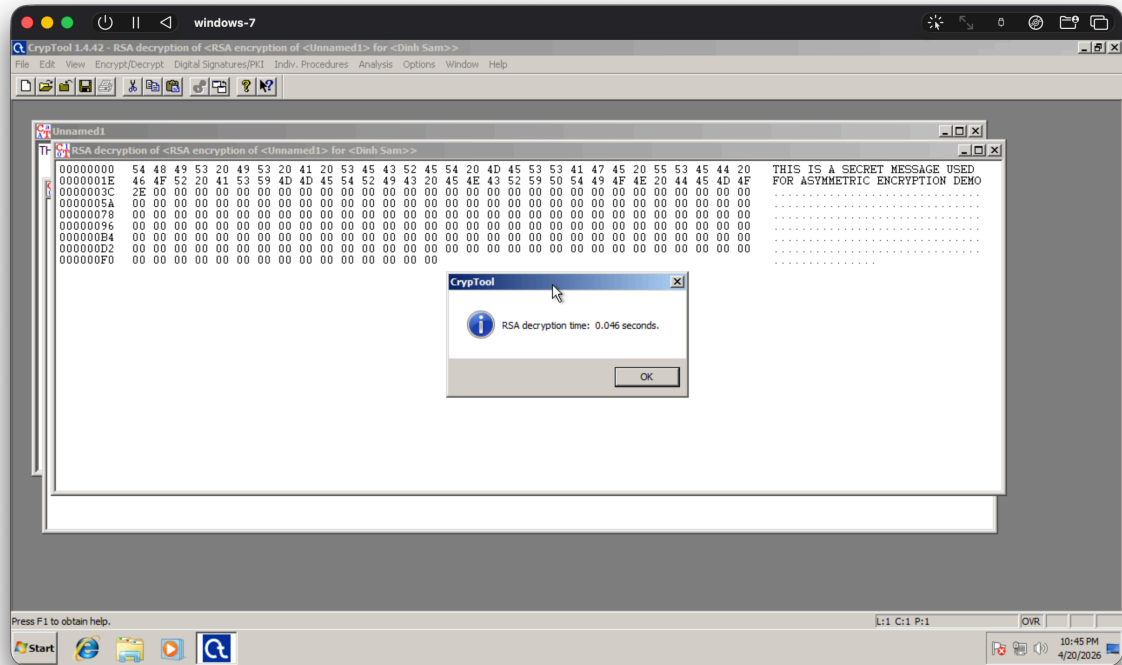
Hình ảnh 26: Bước 3.14. Mã Hóa Văn Bản Với Public Key 2048 Bit

- Giải mã với key RSA 2048 bit: nhập mã PIN đã dùng.



Hình ảnh 27: Bước 3.15. Giải Mã Văn Bản Với Private Key 2048 Bit

- Kết quả giải mã: Hoàn thành trong 0.046 giây.



Hình ảnh 28: Bước 3.16. Văn Bản Giải Mã

- Như vậy, có sự chênh lệch đáng kể khi giải mã với key RSA 1024 bit và 2048 bit.
 - 1024: 0.016 giây

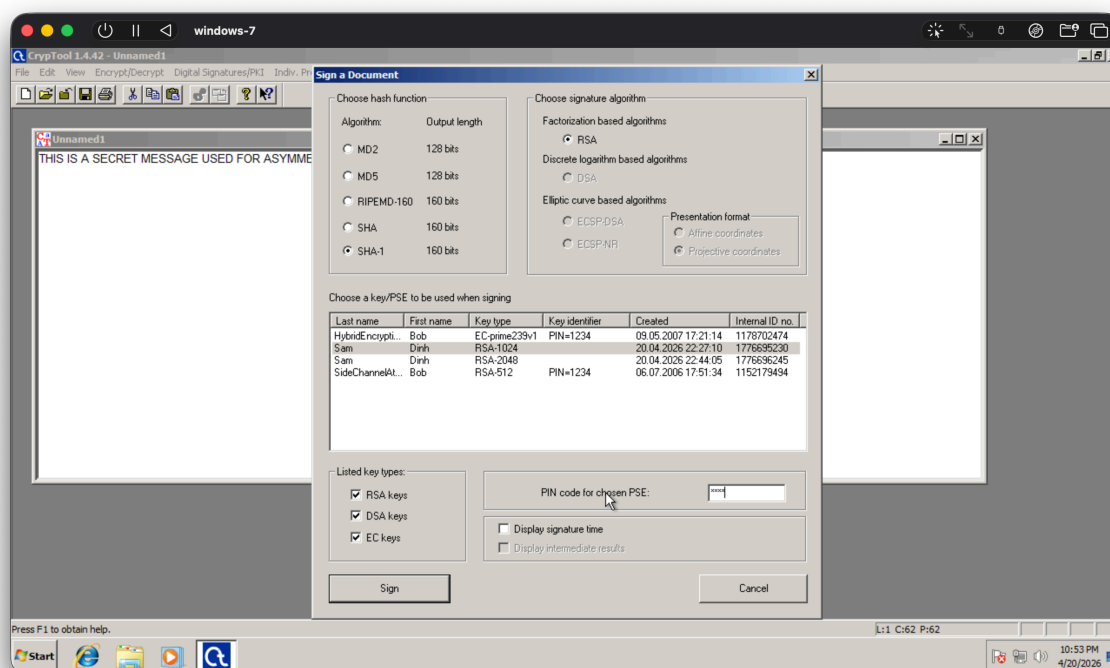
- 2048: 0.046 giây
- Gấp khoảng 3 lần.

CHƯƠNG 4.

BƯỚC 4. CHỮ KÝ SỐ (DIGITAL SIGNATURE)

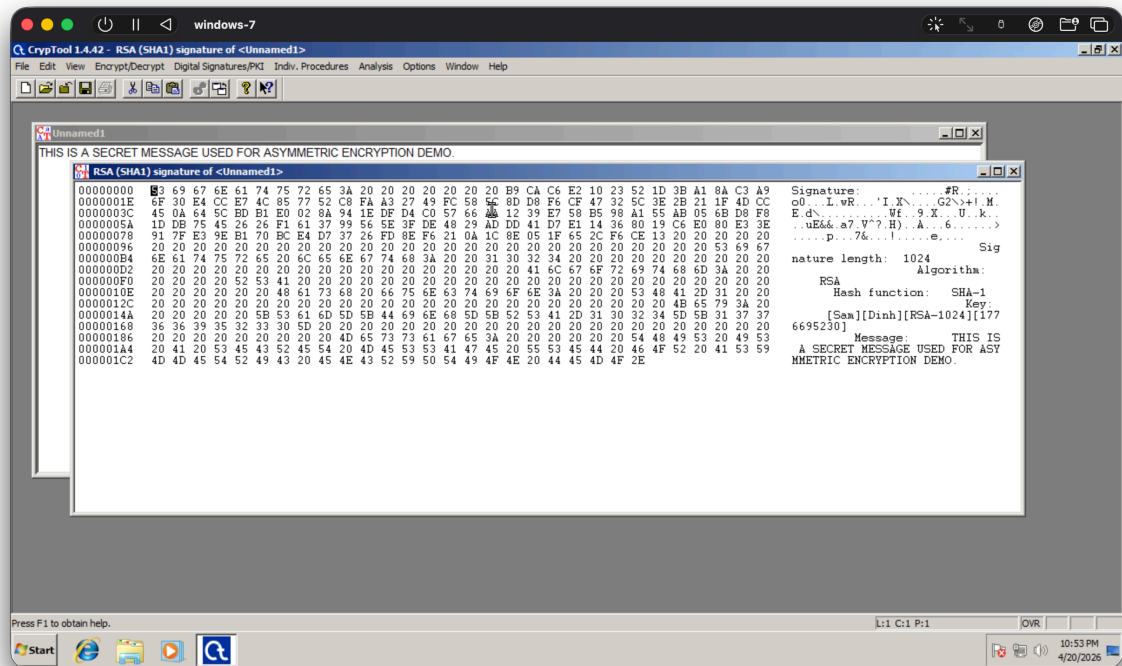
4.1. Sign Document

- Chọn **Digital Signatures/PKI** → **Sign Document...**
 - Hash function: SHA1
 - Algorithms: RSA
 - PSE: Cặp khóa RSA-1024 bit vừa tạo.
 - PIN: 1234 (của cặp khóa vừa tạo).
 - Chọn **Sign**.



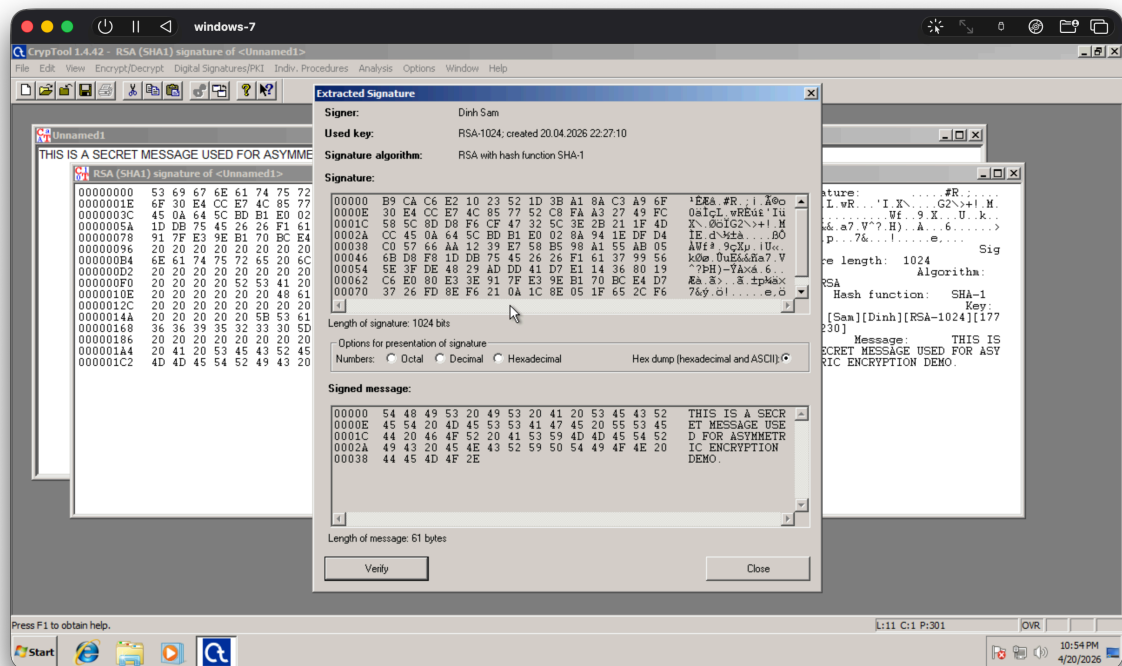
Hình ảnh 29: Bước 4.1. Tạo Chữ Ký Số

- Văn bản đã ký và chữ ký.



Hình ảnh 30: Bước 4.2. Văn Bản Đã Ký Và Chữ Ký

- Xem chữ ký vừa tạo ra.

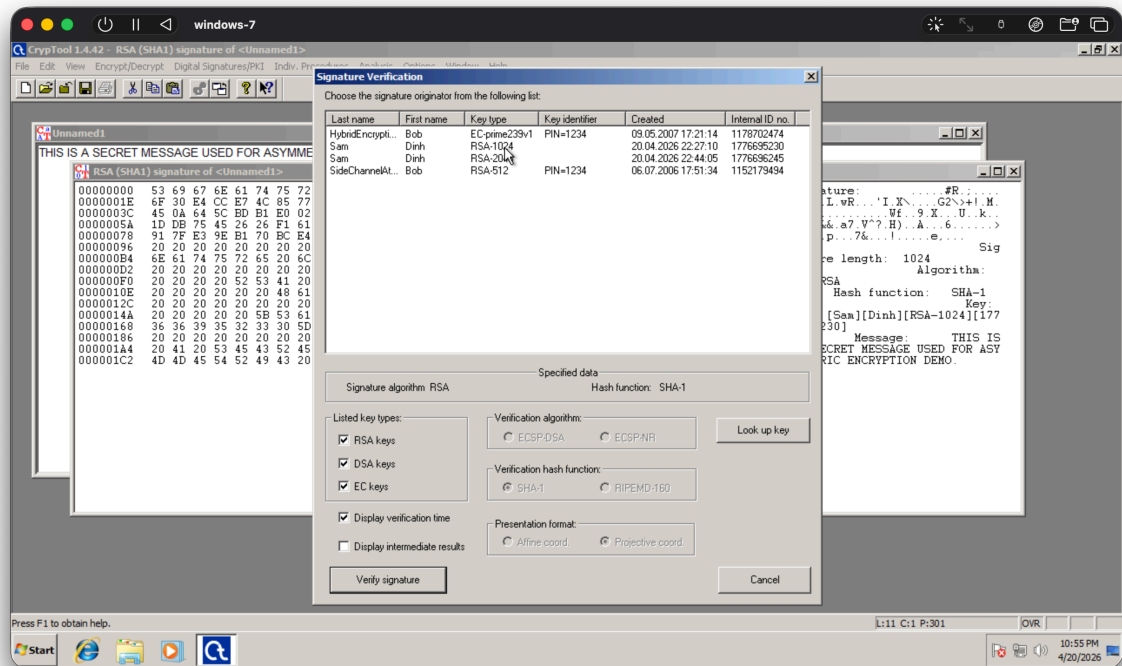


Hình ảnh 31: Bước 4.3. Xem Chữ Ký

4.2. Verify Signature

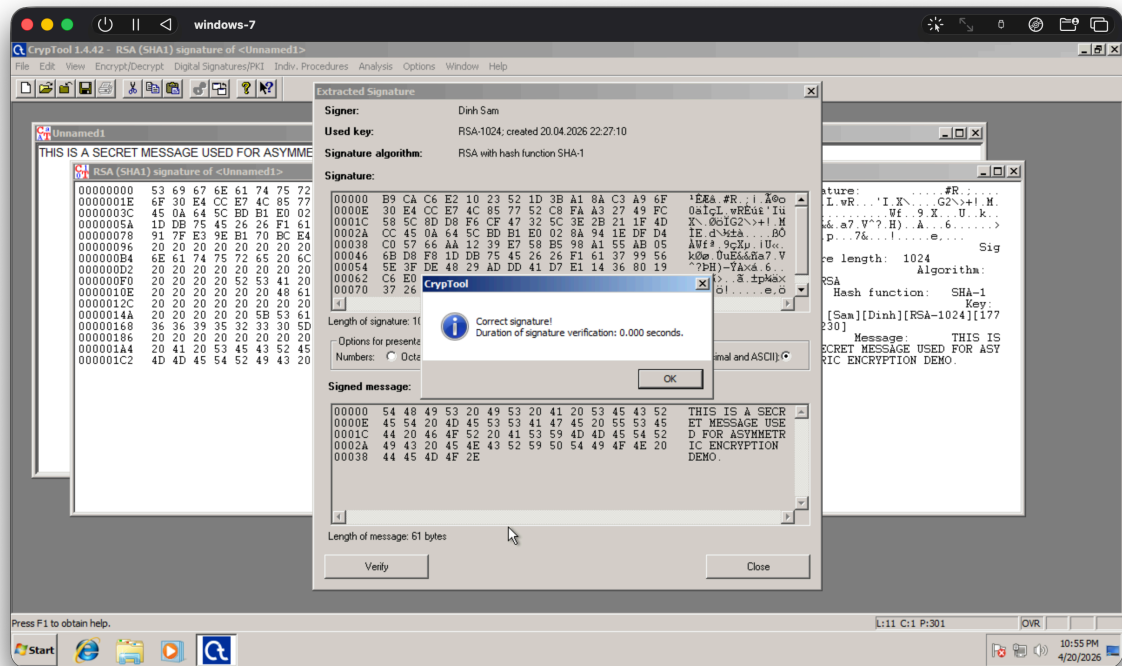
- Chọn Digital Signatures/PKI → Verify Signature...

- ▶ Chọn RSA-1024 vừa tạo.
- ▶ Xác nhận các thông số:
 - Algorithm: RSA
 - Hash function: SHA-1
- ▶ Bấm **Verify signature**



Hình ảnh 32: Bước 4.4. Xác Nhận Chữ Ký

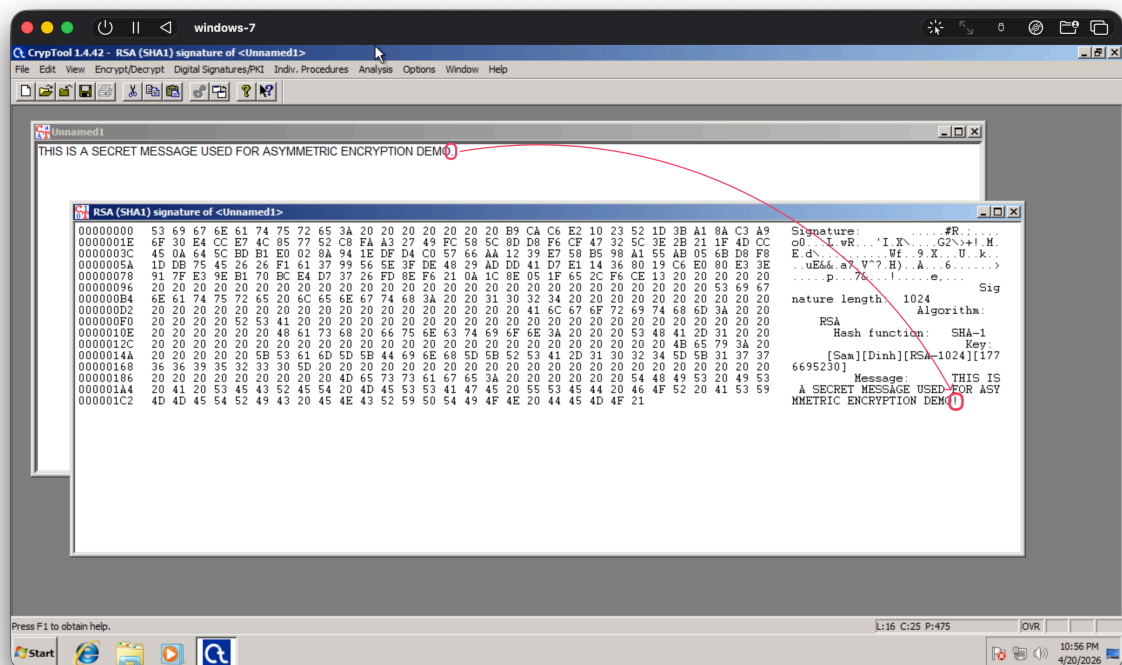
- Kết quả xác nhận chữ ký.
 - ▶ Thành công: chữ ký đúng.



Hình ảnh 33: Bước 4.5. Kết Quả Xác Nhận Chữ Ký

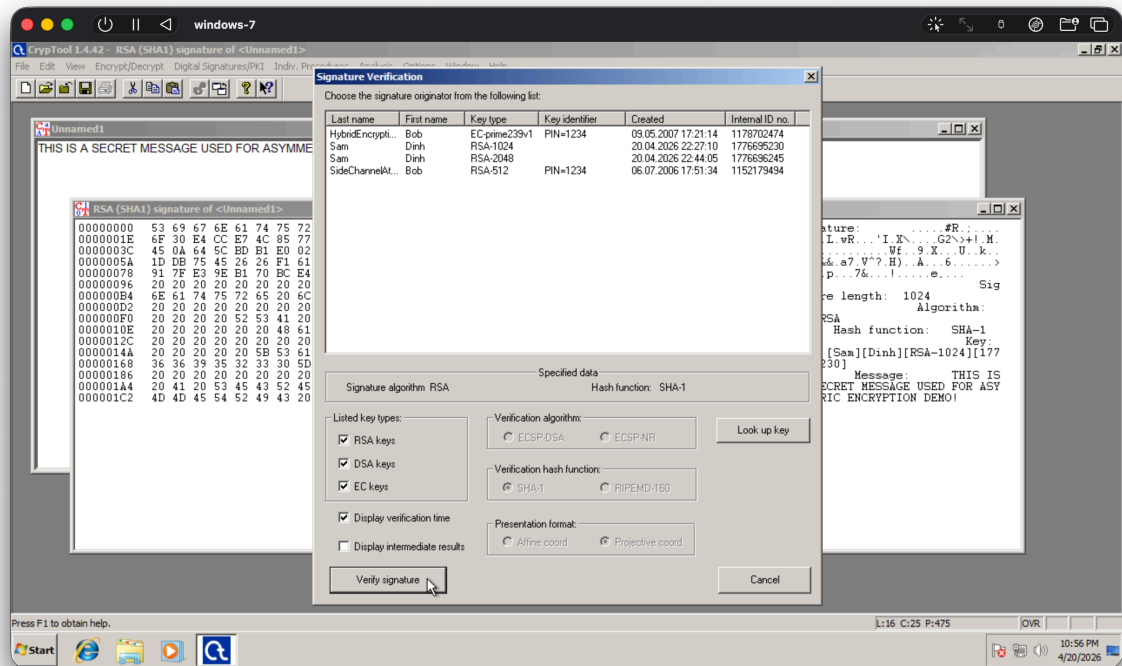
4.3. Giả Lập Chữ Ký Sai

- Thay đổi 1 ký tự trong văn bản và tạo hash lại: Đổi dấu '.' thành dấu '!'.



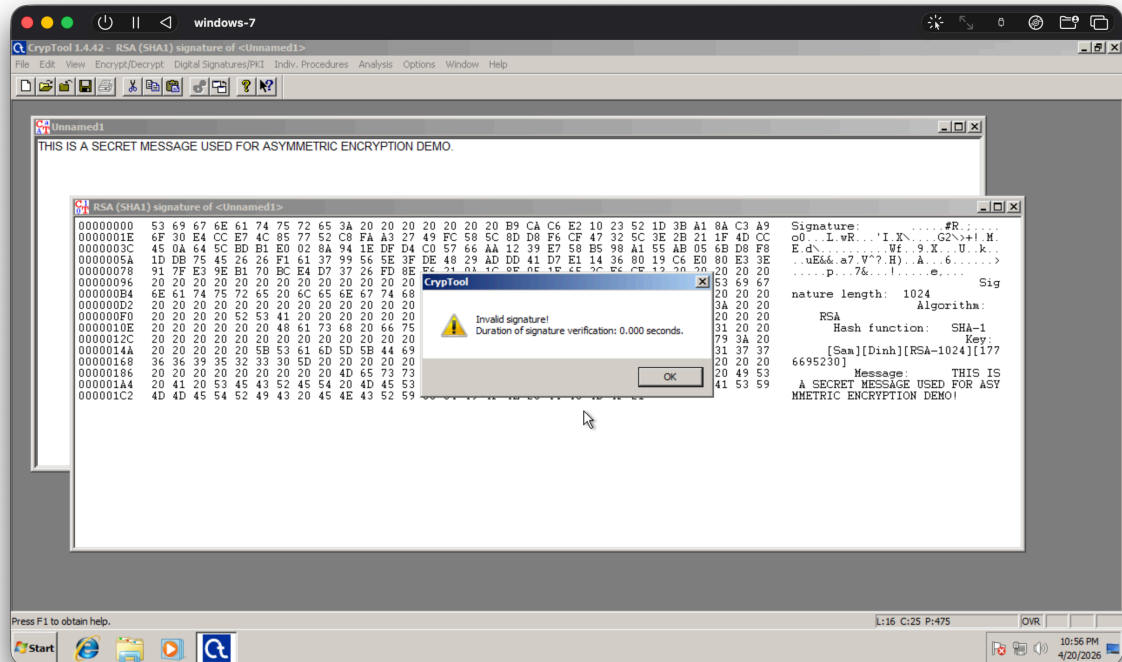
Hình ảnh 34: Bước 4.6. Giả Lập Chữ Ký Sai

- Chạy lại **Verify Signature** với văn bản đã thay đổi.



Hình ảnh 35: Bước 4.7. Chạy Lại Xác Nhận Chữ Ký Sai

- Kết quả của chữ ký đã thay đổi: Sai.



Hình ảnh 36: Bước 4.8. Kết Quả Xác Nhận Chữ Ký Sai

CHƯƠNG 5.

BƯỚC 5. KIỂM TRA TÍNH TOÀN VỆ & XÁC THỰC

- Người gửi gửi văn bản + hash.
- Người nhận tạo hash mới rồi so sánh.
- Nếu hash không khớp → dữ liệu bị thay đổi.

5.1. Văn Bản Gốc Và Hash

- Tạo file văn bản gốc `test.txt` với giá trị:

```
1 THIS IS A SECRET MESSAGE USED FOR ASYMMETRIC ENCRYPTION DEMO.
```

- Tạo giá trị hash cho file văn bản trên: `test.md5`.
- Gửi tới người nhận cả 2 file trên.

```
1 ~/vm 23:13:49
2 $ echo 'THIS IS A SECRET MESSAGE USED FOR ASYMMETRIC ENCRYPTION DEMO.' > test.txt
3
4 ~/vm 23:14:19
5 $ md5sum test.txt
6 d353bd8b4b88befdf24299eaf5c71577 test.txt
7
8 ~/vm 23:14:25
9 $ md5sum test.txt > test.md5
```

5.2. Trường Hợp 1: Văn Bản Nguyên Vẹn

- Người nhận tạo hash cho văn bản đã nhận.
- So sánh với hash của người gửi.
- Không tìm thấy sự khác biệt.
- Như vậy văn bản đã được gửi nguyên vẹn.

```
1 ~/vm 23:14:35
2 $ md5sum test.txt > test-new.md5
3
4 ~/vm 23:14:58
5 $ diff test.md5 test-new.md5
```

5.3. Trường Hợp 2: Văn Bản Bị Thay Đổi

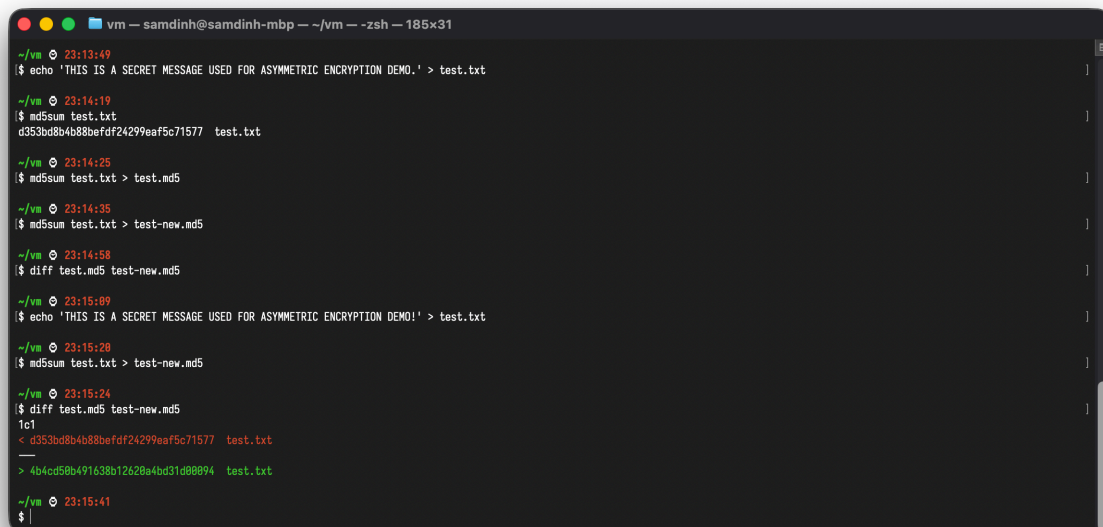
- Vì lý do nào đó, văn bản bị thay đổi trong quá trình truyền.
 - Cụ thể: Thay `.` bằng `!` ở cuối câu.

```
1 ~/vm 23:15:09
2 $ echo 'THIS IS A SECRET MESSAGE USED FOR ASYMMETRIC ENCRYPTION DEMO!' > test.txt
```

- Người nhận tạo hash cho văn bản đã nhận.
- So sánh với hash của người gửi.
- Tìm thấy sự khác biệt.
- Như vậy văn bản đã bị thay đổi.

```
1 ~/vm 23:15:20
2 $ md5sum test.txt > test-new.md5
3
4 ~/vm 23:15:24
5 $ diff test.md5 test-new.md5
6 1c1
7 < d353bd8b4b88befdf24299eaf5c71577 test.txt
8 ---
9 > 4b4cd50b491638b12620a4bd31d00094 test.txt
```

5.4. Demo



```
vm — samdinh@samdinh-mbp — ~/vm — zsh — 185x31

~/vm 23:13:49
$ echo 'THIS IS A SECRET MESSAGE USED FOR ASYMMETRIC ENCRYPTION DEMO.' > test.txt

~/vm 23:14:19
$ md5sum test.txt
d353bd8b4b88befdf24299eaf5c71577 test.txt

~/vm 23:14:25
$ md5sum test.txt > test.md5

~/vm 23:14:35
$ md5sum test.txt > test-new.md5

~/vm 23:14:58
$ diff test.md5 test-new.md5

~/vm 23:15:09
$ echo 'THIS IS A SECRET MESSAGE USED FOR ASYMMETRIC ENCRYPTION DEMO!' > test.txt

~/vm 23:15:20
$ md5sum test.txt > test-new.md5

~/vm 23:15:24
$ diff test.md5 test-new.md5
1c1
< d353bd8b4b88befdf24299eaf5c71577 test.txt
---
> 4b4cd50b491638b12620a4bd31d00094 test.txt

~/vm 23:15:41
$
```

Hình ảnh 37: Bước 5. Kiểm Tra Tính Toàn Vẹn & Xác Thực

CÂU HỎI BÁO CÁO

1. Khác nhau giữa mã hoá đối xứng và bất đối xứng?

- Mã hoá đối xứng (Symmetric Key) là việc cả người gửi và người nhận đều dùng chung một khóa duy nhất ($K_E = K_D$) cho quá trình mã hóa và giải mã, yêu cầu khóa này phải được giữ bí mật tuyệt đối.
- Trong khi đó, mã hoá bất đối xứng (Asymmetric Key) sử dụng một cặp khóa gồm khóa công khai (Public Key) dùng để mã hóa và khóa bí mật (Private Key) dùng để giải mã, tức là hai khóa khác nhau ($K_E \neq K_D$).

2. Hash dùng để làm gì? Vì sao không thể đảo ngược?

- Hàm băm (Hash) dùng để tạo ra một “dấu vân tay kỹ thuật số” (digital digest/fingerprint) có chiều dài cố định từ một thông điệp có chiều dài bất kỳ, phục vụ cho việc chứng thực và kiểm tra tính toàn vẹn của dữ liệu.
- Không thể đảo ngược được hàm băm vì nó có tính chất một chiều (one-way property); thuật toán đã ngắt dữ liệu thành nhiều mảnh, trộn lẫn chúng lại với nhau để tạo ra chuỗi mới, nên việc tính toán ngược lại từ giá trị băm ra dữ liệu gốc là không khả thi.

3. Vì sao sửa 1 ký tự khiến hash đổi toàn bộ?

- Điều này do tính chất kháng đụng độ (collision resistance) hay còn gọi là hiệu ứng tuyết lở (avalanche effect) của hàm băm như được nhắc đến ở Bước 2.
- Bất cứ sự thay đổi nào trên đoạn dữ liệu gốc, dù là rất nhỏ (như sửa 1 ký tự), cũng sẽ làm thuật toán tính toán lại và tạo ra một giá trị băm hoàn toàn khác biệt với tỉ lệ rất cao.

4. Tại sao chữ ký số cần private key?

- Chữ ký số cần khóa bí mật (Private Key) của người gửi để mã hóa chuỗi băm (message digest) của thông điệp.
- Do private key là duy nhất và chỉ có người gửi sở hữu, việc “ký” bằng khóa này đảm bảo rằng người nhận khi dùng Public Key của người gửi để xác minh (verify) có thể tin tưởng tuyệt đối rằng thông điệp thực sự xuất phát từ chính người gửi chứ không phải bị giả mạo bởi ai khác.

5. RSA có dùng để mã hoá file lớn không? Vì sao?

- RSA không thường dùng để mã hóa file lớn.
- Lý do là tốc độ tính toán (mã hóa và giải mã) chậm hơn các giải thuật đối xứng như DES/AES rất rất nhiều.
- Do đó, người ta thường dùng giải thuật đối xứng (như DES) để mã hóa khối văn bản lớn vì tốc độ nhanh, sau đó mới dùng RSA để mã hóa chính cái khóa bí mật của DES đó rồi gửi đi, tức là chỉ một phần văn bản/dữ liệu có kích thước nhỏ nhưng chứa đựng thông tin quan trọng.

6. Chữ ký số khác mã hoá ở điểm nào?

- Mã hóa thông thường dùng để đảm bảo **tính bí mật** (confidentiality), người gửi sẽ dùng Khóa công khai (Public Key) của *người nhận* để mã hóa, và chỉ *người nhận* mới có Khóa bí mật (Private Key) để mở ra xem.
- Trong khi đó, chữ ký số dùng để đảm bảo **tính chứng thực** (authentication), người gửi tự bấm thông điệp và dùng Khóa bí mật (Private Key) của *chính mình* để mã hóa (ký), sau đó ai cũng có thể dùng Khóa công khai (Public Key) của *người gửi* để mở ra xác minh xem có đúng là người đó gửi hay không.

7. Nhược điểm của mode ECB trong AES?

- Nhược điểm của mode ECB (Electronic Codebook) là nếu các khối dữ liệu đầu vào (plaintext) giống hệt nhau, thì khi mã hóa nó sẽ tạo ra các khối mã hóa (ciphertext) giống y hệt nhau.
- Điều này không làm ẩn đi được cấu trúc thống kê của dữ liệu gốc, khiến kẻ tấn công dễ dàng nhận ra các mẫu (pattern) lặp lại và từ đó có thể có các phương án để bẻ khóa.

CHƯƠNG 7.

TỔNG KẾT

- Sinh viên nắm quy trình mã hoá hiện đại.
- Biết sử dụng RSA để mã hoá và ký.
- Thực hành hash để kiểm tra toàn vẹn.
- Hiểu vai trò của khóa công khai & bí mật.
- Áp dụng CrypTool vào an toàn thông tin.

Thông Tin Về Báo Cáo Đây

A.1 Mã Nguồn Báo Cáo

- Miêu tả: Báo cáo này được xây dựng từ mã nguồn [Typst](#), với nội dung chính được chuẩn bị từ Markdown.
- Lưu trữ tại: nhánh [IE105-TH1B](#) của repo [sam-uit/typst-report-template](#)

A.2 Mã Nguồn Nội Dung Chính

- Miêu tả: Được chuẩn bị với Markdown, trong kho của [Obsidian](#).
- Lưu trữ tại: [uit/courses/IE105/assignments](#)

Tài Liệu Tham Khảo
